STARTUB · DIA 2

Frontend Moderno com React e TypeScript

Dia 2 · Formulários, estado e pagamento: a aplicação passa a escrever na API

Evento	StartUB
Encontro	Dia 2
Minicurso	Desenvolvimento Frontend Moderno com React e TypeScript
Professor	Marcos David Souza Ramos
E-mail	marcos.david@prof.unibalsas.edu.br
Carga horária	3 horas
Pré-requisitos	Dia 1 concluído, ou a branch `dia-1` do repositório do minicurso rodando

Centro Universitário Unibalsas · Balsas-MA
Material de apoio do minicurso

Sumário

- Como usar este material
- Roteiro do encontro
- 1. Retomada: quem é dono de cada dado
- 2. Estado local: busca e filtro
- 3. O formulário de inscrição
- 4. Comprovante e minhas inscrições
- 5. O painel do organizador
- 6. Cadastro e login com Zod
- 7. Encerramento
- Para ir além
- Anexo A. O contrato da API
- Anexo B. O mapa do projeto

Como usar este material

O formato é o mesmo do primeiro encontro: siga os **Passos** na ordem, e cada bloco traz o caminho do arquivo e o conteúdo dele **inteiro** naquele momento. Quando o bloco diz **(trecho)**, ele mostra só a parte que muda, e o texto diz onde ela fica.

> **Conceito.** A ideia por trás do que você está fazendo.

> **Na prática.** Como aquilo aparece no trabalho de quem faz frontend.

> **Atenção.** O erro mais comum naquele ponto, com o sintoma que ele produz.

> **CONFIRA ANTES DE SEGUIR.** O que precisa estar funcionando antes do próximo passo.

> **Outro caminho.** Uma alternativa ao que fizemos, para você conhecer.

As respostas da API mostradas aqui vêm de execuções reais, com a conta de teste do material e dois eventos de teste, a **Oficina de Design de APIs**, gratuita, e a **Oficina de Deploy na Nuvem**, paga e com duas vagas. Na sua tela, os eventos e os números serão os da API no dia.

Roteiro do encontro

	Etapa	O que você leva dela
1	**Retomada**	o projeto rodando e um mapa de quem cuida de cada dado
2	**Busca e filtro**	estado derivado: calcular em vez de guardar
3	**O formulário de inscrição**	validação com Zod, envio com `useMutation`, erros do servidor no campo certo
4	**Comprovante e minhas inscrições**	uma página que acompanha o pagamento sozinha
5	**O painel do organizador**	criar, editar e remover eventos, com a vitrine sempre atualizada
6	**Cadastro e login com Zod**	os formulários do Dia 1 reescritos com as ferramentas de hoje
7	**Encerramento**	a versão de produção e o caminho até as equipes do StartUB

1. Retomada: quem é dono de cada dado

Ligue o servidor (`npm run dev`), entre com a sua conta e confira que a vitrine mostra os eventos da API. Quem não terminou o Dia 1 começa pela branch `dia-1` do repositório do minicurso, https://github.com/prof-marcosdavid/startub-eventos, ou pelo arquivo **Ponto de partida do Dia 2.zip**. Depois, `npm install` , `npm run dev` e uma conta em **Criar conta**. O fim de cada etapa de hoje também tem uma tag, de `d2-etapa-2` a `d2-etapa-6` : quem se perder começa uma branch a partir da etapa anterior, com `git switch -c minha-versao d2-etapa-3` , por exemplo.

No primeiro encontro apareceram três lugares para guardar dados: o TanStack Query, para o que vem do servidor; o `useState` , para o que existe só na tela, como os campos do login; e o Context, para o usuário logado, que a aplicação inteira precisa ver. Hoje entra uma quarta ferramenta, o React Hook Form com o Zod, para os formulários maiores. A Figura 1 resume quem cuida de cada coisa. Essa pergunta, **de onde vem este dado e quem é dono dele**, decide qual ferramenta usar em cada passo de hoje.

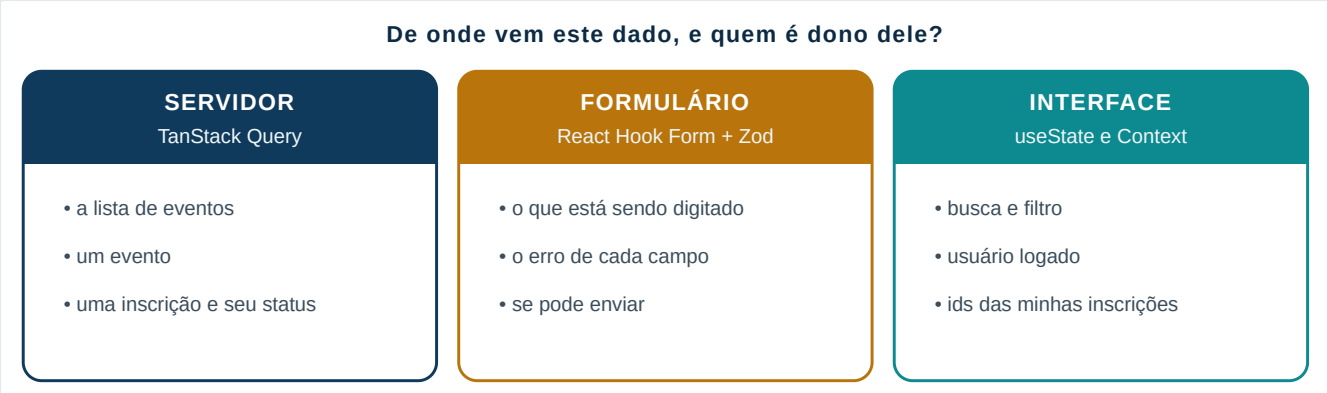


Figura 1. Três tipos de dado, três ferramentas

2. Estado local: busca e filtro

A lista de eventos é dado do servidor. Já o texto digitado na busca e o filtro escolhido existem só na tela, e somem quando a página fecha. Para esse tipo de dado, o React tem o `useState` .

PASSO 2.1 · BUSCA E FILTRO NA VITRINE

Atualize `src/pages/Vitrine.tsx` :

src/pages/Vitrine.tsx

```
import { useState } from "react"
import { useQuery } from "@tanstack/react-query"
import { Search } from "lucide-react"
import { EventoCard } from "@/components/EventoCard"
import { CarregandoCards, MensagemErro, Vazio } from "@/components/Estados"
import { Button } from "@/components/ui/button"
import { Input } from "@/components/ui/input"
import { ehGratuito } from "@/lib/formatar"
```

```tsx
import { listarEventos } from "@/services/eventos"

type Filtro = "todos" | "gratuitos" | "pagos"

const FILTROS: { valor: Filtro; rotulo: string }[] = [
  { valor: "todos", rotulo: "Todos" },
  { valor: "gratuitos", rotulo: "Gratuitos" },
  { valor: "pagos", rotulo: "Pagos" },
]

export function Vitrine() {
  const [busca, setBusca] = useState("")
  const [filtro, setFiltro] = useState<Filtro>("todos")

  const { data: eventos, isPending, error, refetch } = useQuery({
    queryKey: ["eventos"],
    queryFn: listarEventos,
  })

  if (isPending) return <CarregandoCards />
  if (error) return <MensagemErro erro={error} tentarDeNovo={refetch} />

  // Estado derivado: a lista filtrada é calculada a cada renderização,
  // não fica guardada num useState.
  const termo = busca.trim().toLowerCase()
  const visiveis = eventos.filter((evento) => {
    const casaBusca = evento.titulo.toLowerCase().includes(termo)
    const casaFiltro =
      filtro === "todos" ||
      (filtro === "gratuitos" && ehGratuito(evento.valor)) ||
      (filtro === "pagos" && !ehGratuito(evento.valor))
    return casaBusca && casaFiltro
  })

  return (
    <section className="grid gap-8">
      <header className="grid gap-2">
        <h1 className="text-4xl font-extrabold sm:text-5xl">Programação do StartUB 2026</h1>
        <p className="max-w-prose text-lg text-muted-foreground">
          Oficinas, minicursos e o hackathon do congresso. Escolha um evento e faça sua
inscrição.
        </p>
      </header>

      <div className="flex flex-wrap items-center gap-3">
        <div className="relative min-w-60 flex-1">
          <Search className="absolute top-1/2 left-3.5 size-4 -translate-y-1/2 text-muted-
foreground" />
          <Input
            placeholder="Buscar pelo título"
            aria-label="Buscar pelo título"
            className="pl-10"
            value={busca}
            onChange={(e) => setBusca(e.target.value)}
          />
        </div>
```

```
        <div className="flex gap-1 rounded-full bg-muted p-1" role="group" aria-
label="Filtrar por valor">
          {FILTROS.map(({ valor, rotulo }) => (
            <Button
              key={valor}
              size="sm"
              variant={filtro === valor ? "default" : "ghost"}
              aria-pressed={filtro === valor}
              className="rounded-full px-4"
              onClick={() => setFiltro(valor)}
            >
              {rotulo}
            </Button>
          ))}
        </div>
      </div>

      {visiveis.length === 0 ? (
        <Vazio>Nenhum evento com esse título ou filtro. Tente outra busca.</Vazio>
      ) : (
        <div className="grid gap-3">
          {visiveis.map((evento) => (
            <EventoCard key={evento.id} evento={evento} />
          ))}
        </div>
      )}
    </section>
  )
}
```

Os três filtros vêm de uma lista, `FILTROS`, com o valor que o código usa e o rótulo que a pessoa lê. Eles são desenhados como um **grupo de botões**: o escolhido ganha a variante `default`, magenta, e os outros ficam `ghost`. O `aria-pressed` diz ao leitor de tela qual está ativo, e o `aria-label` dá nome ao campo de busca, que não tem rótulo visível.

> **Conceito.** A lista filtrada é **estado derivado**: ela é calculada a partir de `eventos`, `busca` e `filtro` a cada renderização. Guardá-la num terceiro `useState` criaria duas fontes da mesma verdade, que cedo ou tarde ficam diferentes. Regra prática: se dá para calcular, não guarde.

> **Atenção.** Os `useState` e o `useQuery` estão **antes** dos `return` antecipados. Os hooks do React precisam ser chamados sempre, na mesma ordem, em toda renderização. Um `useState` depois de `if (isPending) return ...` funcionaria na primeira vez e quebraria na segunda, com o erro *Rendered more hooks than during the previous render*.

> **CONFIRA ANTES DE SEGUIR.** Digitar parte de um título deixa só os eventos que combinam. **Pagos** mostra só os eventos com valor. Uma busca sem resultado mostra **Nenhum evento encontrado**. Nenhuma requisição nova aparece na aba Network enquanto você filtra: tudo acontece no navegador.

3. O formulário de inscrição

3.1 React Hook Form e Zod

No Dia 1, cada campo do login tinha o seu `useState`, e os erros vinham só da API. Funciona, mas cresce mal: um formulário maior pede um estado por campo, validação antes de enviar, o erro ao lado de cada campo e o botão bloqueado enquanto algo estiver errado. Duas bibliotecas dividem esse trabalho:

- o **React Hook Form** cuida dos campos: liga cada `input` ao formulário com `register` e guarda os valores e os erros, sem redesenhar a página a cada tecla;
- o **Zod** descreve **as regras** num esquema. Do mesmo esquema sai o tipo TypeScript dos dados, de modo que regra e tipo nunca ficam diferentes.

A API tem as suas regras, e o formulário deve conferir as mesmas antes de enviar. A Tabela 1 põe as duas lado a lado.

Campo	Regra no Laravel	Regra no Zod
nome	`required`, `max:120`	`z.string().trim().min(3).max(120)`
email	`required`, `email`, único no evento	`z.email()`
cpf	`required`, `digits:11`	`z.string().regex(/^\d{11}$/)`

> **Conceito.** Validar no navegador é **conforto**: o erro aparece na hora, sem esperar a rede. Validar no servidor é **segurança**: qualquer um pode chamar a API sem passar pelo seu formulário. Uma coisa não substitui a outra. E há regras que só o servidor consegue conferir, como "este e-mail já está inscrito".

PASSO 3.1 · OS TIPOS DA INSCRIÇÃO

Atualize `src/tipos.ts` com o formato da inscrição:

src/tipos.ts

```ts
// O formato dos dados, copiado das respostas reais da API.

export interface Evento {
  id: number
  titulo: string
  descricao: string | null
  local: string
  data: string // "2026-12-05 19:00", sem o T
  vagas: number
  valor: string // "25.00", texto e não número
  created_at: string
  updated_at: string
  user_id: number | null // quem criou; null nos eventos de exemplo da API
}

export type StatusInscricao = "pendente" | "confirmada" | "cancelada"
```

```ts
export interface Inscricao {
  id: number
  evento_id: number
  nome: string
  email: string
  cpf: string
  status: StatusInscricao
  link_pagamento?: string | null
  pago_em?: string | null
  created_at: string
  user_id: number // quem se inscreveu
  evento?: Evento // só vem na consulta de uma inscrição
}

export interface Usuario {
  id: number
  name: string
  email: string
}
```

PASSO 3.2 · O ESQUEMA DA INSCRIÇÃO

Crie a pasta `src/schemas` e, nela, `inscricao.ts`:

`src/schemas/inscricao.ts`

```ts
import { z } from "zod"

// As mesmas regras que a API aplica no Laravel:
// nome obrigatório até 120, e-mail válido, CPF com 11 dígitos.
export const esquemaInscricao = z.object({
  nome: z
    .string()
    .trim()
    .min(3, "Informe seu nome completo.")
    .max(120, "Use no máximo 120 caracteres."),
  email: z.email("Informe um e-mail válido."),
  cpf: z.string().regex(/^\d{11}$/, "Use os 11 números do CPF, sem pontos nem traço."),
})

export type DadosInscricao = z.infer<typeof esquemaInscricao>
```

O `z.infer` lê o esquema e produz o tipo `{ nome: string; email: string; cpf: string }`. Ninguém escreve esse tipo à mão.

PASSO 3.3 · O SERVIÇO DE INSCRIÇÕES

Crie `src/services/inscricoes.ts`:

`src/services/inscricoes.ts`

```ts
import type { Inscricao } from "@/tipos"
import type { DadosInscricao } from "@/schemas/inscricao"
import { requisicao } from "./api"
```

```ts
export function inscrever(eventoId: number, dados: DadosInscricao) {
  return requisicao<Inscricao>(`/eventos/${eventoId}/inscricoes`, {
    method: "POST",
    body: JSON.stringify(dados),
  })
}

export function buscarInscricao(id: number) {
  return requisicao<Inscricao>(`/inscricoes/${id}`)
}
```

PASSO 3.4 · O FORMULÁRIO

Crie `src/components/FormularioInscricao.tsx`:

src/components/FormularioInscricao.tsx

```tsx
import { useForm } from "react-hook-form"
import { zodResolver } from "@hookform/resolvers/zod"
import { useMutation } from "@tanstack/react-query"
import { toast } from "sonner"
import { Button } from "@/components/ui/button"
import { Card, CardContent, CardDescription, CardHeader, CardTitle } from
"@/components/ui/card"
import { Field, FieldError, FieldGroup, FieldLabel } from "@/components/ui/field"
import { Input } from "@/components/ui/input"
import { useAuth } from "@/contexts/AuthContext"
import { ehGratuito, formatarValor } from "@/lib/formatar"
import { esquemaInscricao, type DadosInscricao } from "@/schemas/inscricao"
import { inscrever } from "@/services/inscricoes"
import type { Evento } from "@/tipos"

export function FormularioInscricao({ evento }: { evento: Evento }) {
  const { usuario } = useAuth()

  const {
    register,
    handleSubmit,
    reset,
    formState: { errors },
  } = useForm<DadosInscricao>({
    resolver: zodResolver(esquemaInscricao),
    // nome e e-mail já vêm da conta; a pessoa só confere e completa o CPF
    defaultValues: { nome: usuario?.name ?? "", email: usuario?.email ?? "", cpf: "" },
  })

  const inscricao = useMutation({
    mutationFn: (dados: DadosInscricao) => inscrever(evento.id, dados),
    onSuccess: (nova) => {
      toast.success(`Inscrição nº ${nova.id} realizada!`)
      reset()
    },
  })

  return (
```

```
    <Card className="h-fit lg:sticky lg:top-24">
      <CardHeader>
        <CardTitle className="text-2xl font-bold">Sua inscrição</CardTitle>
        <CardDescription>
          {ehGratuito(evento.valor)
            ? "Inscrição gratuita. Confira seus dados e confirme."
            : `Inscrição de ${formatarValor(evento.valor)}. O pagamento abre depois que você
confirmar.`}
        </CardDescription>
      </CardHeader>
      <CardContent>
        <form onSubmit={handleSubmit((dados) => inscricao.mutate(dados))} noValidate>
          <FieldGroup>
            <Field data-invalid={!!errors.nome}>
              <FieldLabel htmlFor="nome">Nome completo</FieldLabel>
              <Input id="nome" aria-invalid={!!errors.nome} {...register("nome")} />
              <FieldError errors={[errors.nome]} />
            </Field>

            <Field data-invalid={!!errors.email}>
              <FieldLabel htmlFor="email">E-mail</FieldLabel>
              <Input id="email" type="email" aria-invalid={!!errors.email}
{...register("email")} />
              <FieldError errors={[errors.email]} />
            </Field>

            <Field data-invalid={!!errors.cpf}>
              <FieldLabel htmlFor="cpf">CPF</FieldLabel>
              <Input
                id="cpf"
                inputMode="numeric"
                maxLength={11}
                placeholder="Somente números"
                aria-invalid={!!errors.cpf}
                {...register("cpf")}
              />
              <FieldError errors={[errors.cpf]} />
            </Field>

            <Button type="submit" disabled={inscricao.isPending}>
              {inscricao.isPending ? "Enviando..." : "Confirmar inscrição"}
            </Button>
          </FieldGroup>
        </form>
      </CardContent>
    </Card>
  )
}
```

As peças, de cima para baixo:

- `useForm` cria o formulário. O `resolver` entrega a validação ao Zod, e `defaultValues` já preenche o nome e o e-mail com os da conta, lidos do `useAuth()`;

- `{...register("cpf")}` liga o `Input` ao campo `cpf`. O que o React Hook Form precisa, como `name`, `onChange` e `ref`, entra no input por esse espalhamento. Compare com o `value` e o `onChange` escritos à mão no login do Dia 1;
- `handleSubmit` só chama a função de envio **se o Zod aprovar**. Se reprovar, preenche `errors` e nada vai para a API;
- `useMutation` é o par do `useQuery` para **escrever**: chama a função quando mandamos (`mutate`) e informa `isPending` enquanto a resposta não chega, o que desabilita o botão;
- `reset()` volta os campos aos valores iniciais depois do sucesso;
- a descrição do cartão muda com o evento: gratuita, ou com o valor e o aviso de que o pagamento vem depois. Escrever o que vai acontecer evita surpresa na hora de pagar.

> **Na prática.** O `noValidate` no `<form>` desliga a validação nativa do navegador, aqueles balões que cada navegador desenha de um jeito. A validação fica toda com o Zod, com as mensagens que escrevemos.

PASSO 3.5 · MOSTRAR O FORMULÁRIO E OS AVISOS

Atualize `src/pages/DetalheEvento.tsx`, que passa a ter duas colunas, o evento e o formulário:

src/pages/DetalheEvento.tsx

```
import { Link, useParams } from "react-router"
import { useQuery } from "@tanstack/react-query"
import { ArrowLeft } from "lucide-react"
import { DataDoEvento } from "@/components/DataDoEvento"
import { FormularioInscricao } from "@/components/FormularioInscricao"
import { MensagemErro } from "@/components/Estados"
import { Badge } from "@/components/ui/badge"
import { Skeleton } from "@/components/ui/skeleton"
import { ehGratuito, formatarData, formatarValor } from "@/lib/formatar"
import { buscarEvento } from "@/services/eventos"

export function DetalheEvento() {
  const { id } = useParams()
  const eventoId = Number(id)

  const { data: evento, isPending, error } = useQuery({
    queryKey: ["eventos", eventoId],
    queryFn: () => buscarEvento(eventoId),
  })

  if (isPending) return <Skeleton className="h-96 rounded-xl" />
  if (error) return <MensagemErro erro={error} />

  return (
    <div className="grid gap-10 lg:grid-cols-[1fr_24rem] lg:items-start">
      <article className="grid gap-6">
        <Link to="/" className="flex w-fit items-center gap-1 text-sm text-muted-foreground
hover:text-foreground">
          <ArrowLeft className="size-4" /> Voltar para a programação
        </Link>
```

```
        <div className="flex items-start gap-6">
          <DataDoEvento data={evento.data} className="pt-1" />
          <div className="grid gap-3">
            <Badge variant={ehGratuito(evento.valor) ? "secondary" : "default"}>
              {formatarValor(evento.valor)}
            </Badge>
            <h1 className="text-4xl leading-none font-extrabold sm:text-5xl">{evento.titulo}
</h1>
          </div>
        </div>

        {evento.descricao && <p className="max-w-prose text-lg text-muted-foreground">
{evento.descricao}</p>}

        <dl className="grid gap-4 rounded-xl bg-muted p-5 sm:grid-cols-3">
          <div>
            <dt className="text-sm text-muted-foreground">Quando</dt>
            <dd className="font-semibold">{formatarData(evento.data)}</dd>
          </div>
          <div>
            <dt className="text-sm text-muted-foreground">Onde</dt>
            <dd className="font-semibold">{evento.local}</dd>
          </div>
          <div>
            <dt className="text-sm text-muted-foreground">Vagas</dt>
            <dd className="font-semibold">{evento.vagas}</dd>
          </div>
        </dl>
      </article>

      <FormularioInscricao evento={evento} />
    </div>
  )
}
```

E acrescente o `Toaster`, que desenha os avisos, em `src/main.tsx`:

`src/main.tsx`

```
import { StrictMode } from "react"
import { createRoot } from "react-dom/client"
import { BrowserRouter } from "react-router"
import { QueryClient, QueryClientProvider } from "@tanstack/react-query"
import { ReactQueryDevtools } from "@tanstack/react-query-devtools"

import "./index.css"
import App from "./App.tsx"
import { ThemeProvider } from "@/components/theme-provider.tsx"
import { Toaster } from "@/components/ui/sonner"
import { AuthProvider } from "@/contexts/AuthContext"

const queryClient = new QueryClient({
  defaultOptions: {
    queries: { staleTime: 30_000 }, // dados valem 30 s antes de buscar de novo
  },
})
```

```
createRoot(document.getElementById("root")!).render(
  <StrictMode>
    <ThemeProvider>
      <QueryClientProvider client={queryClient}>
        <BrowserRouter>
          <AuthProvider>
            <App />
          </AuthProvider>
        </BrowserRouter>
        <Toaster richColors position="top-center" />
        <ReactQueryDevtools />
      </QueryClientProvider>
    </ThemeProvider>
  </StrictMode>
)
```

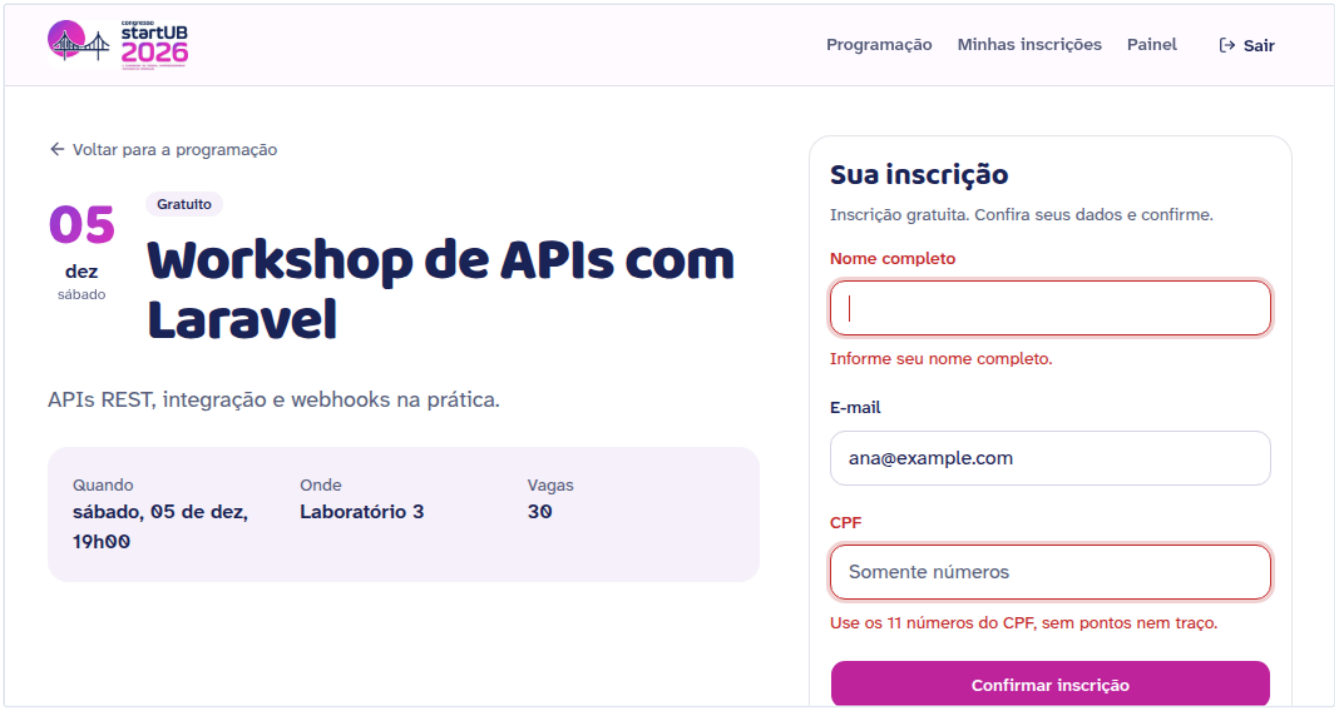


Figura 2. Enviar com o CPF vazio: o Zod barra antes de qualquer requisição

> **CONFIRA ANTES DE SEGUIR.** O formulário já abre com o seu nome e o seu e-mail. Apagar o nome e clicar em **Confirmar inscrição** com o CPF vazio mostra as mensagens em vermelho, e a aba Network não registra requisição nenhuma. Preenchido corretamente num evento gratuito, aparece o aviso **Inscrição nº ... realizada!**, com o número que a API deu.

3.2 Quando o servidor diz não

O Zod aprovou, a requisição saiu, e mesmo assim a API pode recusar. Repita a inscrição com o mesmo e-mail no mesmo evento. Antes, a primeira inscrição, aceita:

POST https://startub.appmake.com.br/api/eventos/12/inscricoes 201 Created

corpo enviado

```
{
  "nome": "Material do Minicurso",
  "email": "material.minicurso@example.com",
  "cpf": "52998224725"
}
```

RESPOSTA


```
{
  "nome": "Material do Minicurso",
  "email": "material.minicurso@example.com",
  "cpf": "52998224725",
  "user_id": 1,
  "status": "confirmada",
  "evento_id": 12,
  "updated_at": "2026-11-12T20:45:19.000000Z",
  "created_at": "2026-11-12T20:45:19.000000Z",
  "id": 5
}
```


A segunda, com o mesmo e-mail:

POST https://startub.appmake.com.br/api/eventos/12/inscricoes 422 Unprocessable Content

corpo enviado

```
{
  "nome": "Material do Minicurso",
  "email": "material.minicurso@example.com",
  "cpf": "52998224725"
}
```

RESPOSTA


```
{
  "message": "The email has already been taken.",
  "errors": {
    "email": [
      "The email has already been taken."
    ]
  }
}
```


O status **422** traz em `errors` um objeto com uma lista de mensagens **por campo**, com os mesmos nomes de campo do nosso formulário. E as mensagens vêm em inglês. Quando vários campos falham, vêm todos de uma vez:

POST https://startub.appmake.com.br/api/eventos/12/inscricoes 422 Unprocessable Content

corpo enviado

```
{
  "nome": "",
  "email": "ana",
  "cpf": "123"
}
```

RESPOSTA

```
{
  "message": "The nome field is required. (and 2 more errors)",
  "errors": {
    "nome": [
      "The nome field is required."
    ],
    "email": [
      "The email field must be a valid email address."
    ],
    "cpf": [
      "The cpf field must be 11 digits."
    ]
  }
}
```

Há também recusas que não são de um campo. Um evento com duas vagas, depois de duas inscrições:

POST https://startub.appmake.com.br/api/eventos/13/inscricoes 201 Created

corpo enviado

```
{
  "nome": "Bruno Lima",
  "email": "bruno@example.com",
  "cpf": "11144477735"
}
```

POST https://startub.appmake.com.br/api/eventos/13/inscricoes 201 Created

corpo enviado

```
{
  "nome": "Carla Dias",
  "email": "carla@example.com",
  "cpf": "11144477735"
}
```

POST https://startub.appmake.com.br/api/eventos/13/inscricoes 409 Conflict

corpo enviado

```
{
  "nome": "Daniel Reis",
  "email": "daniel@example.com",
```

```
  "cpf": "11144477735"
}
```

RESPOSTA

```
{
  "message": "Não há vagas para este evento."
}
```

A Figura 3 mostra o destino de cada resposta: o 422 vai para os campos, o resto vira aviso.

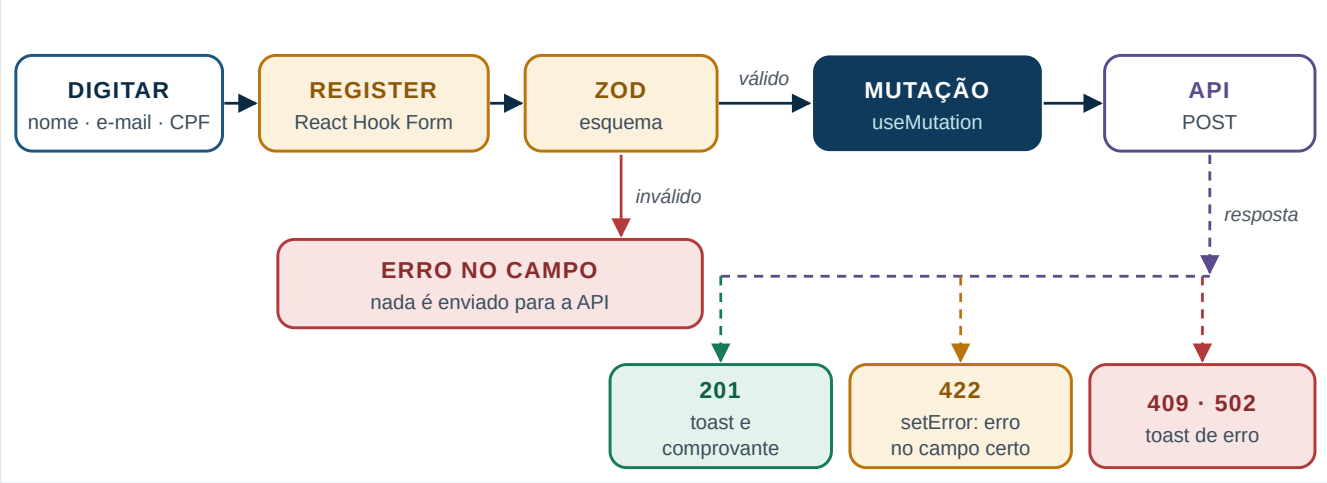


Figura 3. O caminho de um envio, e para onde vai cada resposta

PASSO 3.6 · DISTRIBUIR OS ERROS DA API PELOS CAMPOS

Atualize `src/lib/erros.ts`. A tabela de traduções ganha as mensagens da inscrição, e entra a função `aplicarErrosDaApi`:

src/lib/erros.ts

```
import type { FieldValues, Path, UseFormSetError } from "react-hook-form"
import { ApiError } from "@/services/api"

// A API responde as validações em inglês. Esta tabela traduz as mensagens
// que os formulários podem receber. O que não estiver aqui aparece como veio.
const TRADUCOES: [RegExp, string][] = [
  [/has already been taken/, "Este e-mail já está em uso."],
  [/field is required/, "Campo obrigatório."],
  [/must be a valid email/, "Informe um e-mail válido."],
  [/must be (\d+) digits/, "Use $1 números, sem pontos nem traço."],
  [/must be a date after now/, "A data precisa ser futura."],
  [/must be at least (\d+) characters/, "Use pelo menos $1 caracteres."],
  [/must be at least (\d+)/, "O valor mínimo é $1."],
  [/must not be greater than (\d+) characters/, "Use no máximo $1 caracteres."],
  [/confirmation does not match/, "As senhas não conferem."],
  [/Too Many Attempts/, "Muitas tentativas. Espere um minuto e tente de novo."],
  [/This action is unauthorized/, "Você não tem permissão para isso: só quem criou pode
alterar."],
  [/No query results/, "Não encontramos o que você procurou."],
]
```

```ts
export function traduzir(mensagem: string): string {
  for (const [padrao, traducao] of TRADUCOES) {
    const achou = mensagem.match(padrao)
    if (achou) return traducao.replace("$1", achou[1] ?? "")
  }
  return mensagem
}

// Leva cada erro 422 da API para o campo correspondente do formulário.
// Devolve true se havia erros de campo para mostrar.
export function aplicarErrosDaApi<T extends FieldValues>(
  erro: unknown,
  setError: UseFormSetError<T>,
): boolean {
  if (!(erro instanceof ApiError) || erro.status !== 422) return false
  for (const [campo, mensagens] of Object.entries(erro.erros)) {
    setError(campo as Path<T>, { message: traduzir(mensagens[0]) })
  }
  return true
}
```

O `aplicarErrosDaApi` é **genérico**: serve para qualquer formulário do React Hook Form, porque recebe o `setError` de quem o chama. Vamos reutilizá-lo no painel e no cadastro.

PASSO 3.7 · USAR NO FORMULÁRIO

Em `src/components/FormularioInscricao.tsx`, acrescente o import junto dos outros:

`src/components/FormularioInscricao.tsx (trecho)`

```ts
import { aplicarErrosDaApi, traduzir } from "@/lib/erros"
```

e troque o começo da função até o fim do `useMutation`, que passa a pegar o `setError` e a ter o `onError`:

`src/components/FormularioInscricao.tsx (trecho)`

```ts
export function FormularioInscricao({ evento }: { evento: Evento }) {
  const { usuario } = useAuth()

  const {
    register,
    handleSubmit,
    setError,
    reset,
    formState: { errors },
  } = useForm<DadosInscricao>({
    resolver: zodResolver(esquemaInscricao),
    // nome e e-mail já vêm da conta; a pessoa só confere e completa o CPF
    defaultValues: { nome: usuario?.name ?? "", email: usuario?.email ?? "", cpf: "" },
  })

  const inscricao = useMutation({
```

```
    mutationFn: (dados: DadosInscricao) => inscrever(evento.id, dados),
    onSuccess: (nova) => {
      toast.success(`Inscrição nº ${nova.id} realizada!`)
      reset()
    },
    onError: (erro) => {
      // 422: erros de campo. Qualquer outro (409, 502, rede): aviso geral.
      if (!aplicarErrosDaApi(erro, setError)) toast.error(traduzir(erro.message))
    },
```

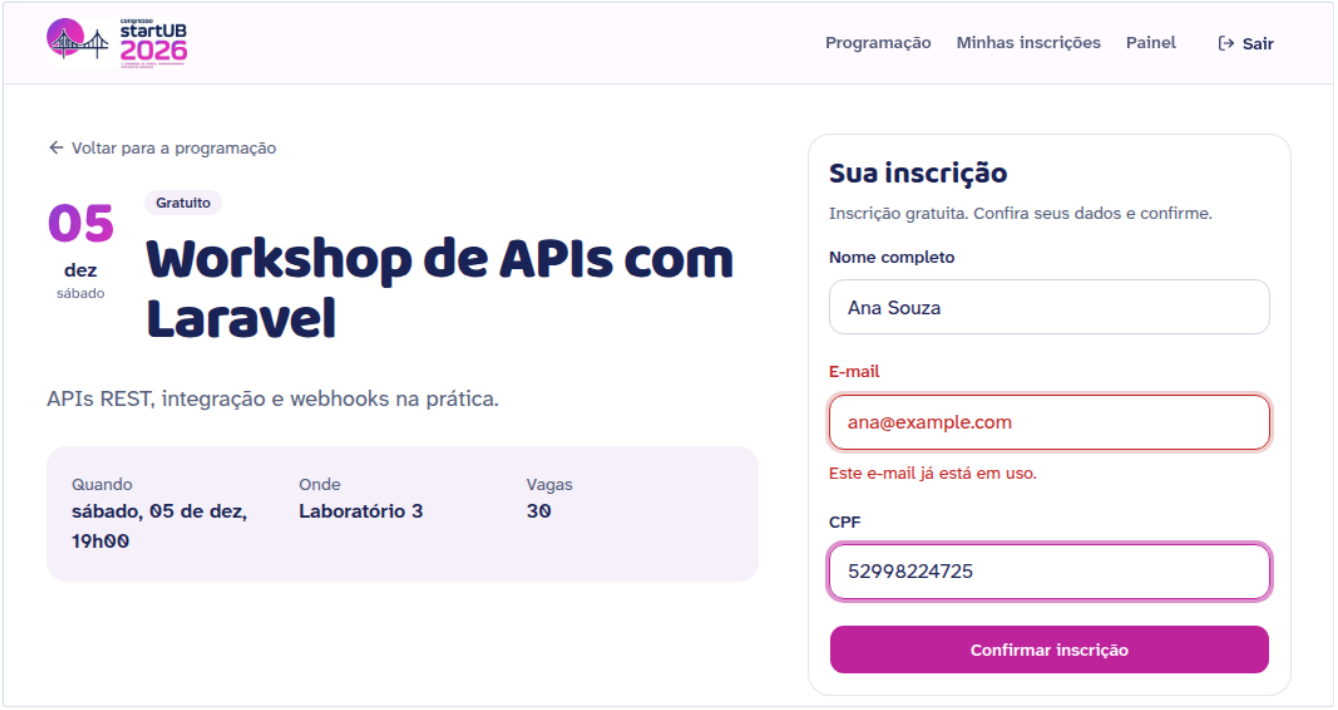


Figura 4. O 422 da API, traduzido, no campo certo

> **CONFIRA ANTES DE SEGUIR.** Inscrever duas vezes o mesmo e-mail no mesmo evento mostra **Este e-mail já está em uso.** embaixo do campo de e-mail. No evento de Git, que tem duas vagas, as inscrições da turma esgotam a vaga depressa, e a seguinte mostra o aviso **Não há vagas para este evento.** no topo da tela.

4. Comprovante e minhas inscrições

4.1 O pagamento acontece fora da aplicação

Quando o evento é pago, a inscrição nasce **pendente**, com um link de pagamento do Asaas. Esta é a consulta da inscrição da Carla, a última aceita no evento pago da seção anterior:

GET https://startub.appmake.com.br/api/inscricoes/7 200 OK

RESPOSTA


```
{
  "id": 7,
  "evento_id": 13,
  "nome": "Carla Dias",
  "email": "carla@example.com",
  "cpf": "11144477735",
  "status": "pendente",
  "created_at": "2026-11-12T20:45:23.000000Z",
  "updated_at": "2026-11-12T20:45:24.000000Z",
  "asaas_cliente_id": "cus_000009949324",
  "asaas_cobranca_id": "pay_0_d0jon24r2cw2t9",
  "link_pagamento": "https://sandbox.asaas.com/i/0_d0jon24r2cw2t9",
  "pago_em": null,
  "user_id": 1,
  "evento": {
    "id": 13,
    "titulo": "Oficina de Deploy na Nuvem",
    "descricao": "Publicando uma aplicação do zero.",
    "local": "Laboratório 4",
    "data": "2026-12-16 19:00",
    "vagas": 2,
    "valor": "25.00",
    "created_at": "2026-11-12T20:45:19.000000Z",
    "updated_at": "2026-11-12T20:45:19.000000Z",
    "user_id": 1
  }
}
```


O pagamento acontece na página do Asaas, fora da nossa aplicação. Quando ele é confirmado, o Asaas avisa a API por um **webhook**, e a API muda o status para `confirmada`. A nossa aplicação não recebe esse aviso: ela só fica sabendo se **perguntar de novo**. A Figura 5 mostra a conversa inteira.

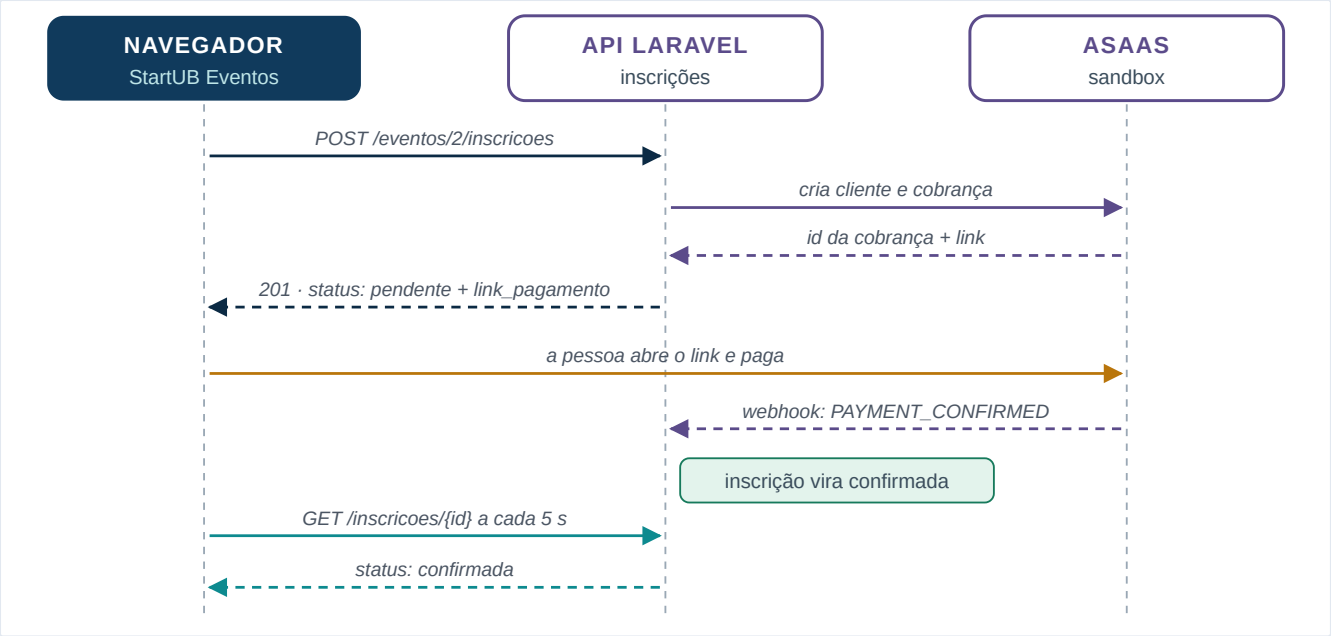


Figura 5. O pagamento: a API fica sabendo pelo webhook, a aplicação, perguntando

> **Conceito.** Perguntar de tempos em tempos se algo mudou chama-se **polling**. O TanStack Query faz isso com uma opção, `refetchInterval`, que aceita uma função: assim dá para perguntar **só enquanto** o status for `pendente`, e parar quando confirmar.

PASSO 4.1 · A ETIQUETA DE STATUS

Crie `src/components/StatusBadge.tsx`:

src/components/StatusBadge.tsx

```tsx
import { Badge } from "@/components/ui/badge"
import type { StatusInscricao } from "@/tipos"

const ROTULOS: Record<StatusInscricao, string> = {
  pendente: "Aguardando pagamento",
  confirmada: "Confirmada",
  cancelada: "Cancelada",
}

const VARIANTES = {
  pendente: "outline",
  confirmada: "default",
  cancelada: "destructive",
} as const

export function StatusBadge({ status }: { status: StatusInscricao }) {
  return <Badge variant={VARIANTES[status]}>{ROTULOS[status]}</Badge>
}
```

O `Record<StatusInscricao, string>` obriga a tabela a ter uma entrada para **cada** status. Se a API ganhar um status novo e ele for acrescentado ao tipo, o TypeScript aponta esta tabela como incompleta.

PASSO 4.2 · A PÁGINA DO COMPROVANTE

Crie `src/pages/Comprovante.tsx`:

src/pages/Comprovante.tsx

```
import { Link, useParams } from "react-router"
import { useQuery } from "@tanstack/react-query"
import { ExternalLink, LoaderCircle } from "lucide-react"
import { DataDoEvento } from "@/components/DataDoEvento"
import { MensagemErro } from "@/components/Estados"
import { StatusBadge } from "@/components/StatusBadge"
import { Button } from "@/components/ui/button"
import { Skeleton } from "@/components/ui/skeleton"
import { formatarValor, partesDaData } from "@/lib/formatar"
import { buscarInscricao } from "@/services/inscricoes"

export function Comprovante() {
  const { id } = useParams()
  const inscricaoId = Number(id)

  const { data: inscricao, isPending, error } = useQuery({
    queryKey: ["inscricoes", inscricaoId],
    queryFn: () => buscarInscricao(inscricaoId),
    // Enquanto o pagamento não cai, pergunta de novo à API a cada 5 segundos.
    refetchInterval: (query) => (query.state.data?.status === "pendente" ? 5000 : false),
  })

  if (isPending) return <Skeleton className="mx-auto h-96 max-w-md rounded-2xl" />
  if (error) return <MensagemErro erro={error} />

  const { evento } = inscricao

  // A credencial do congresso: o evento em cima, o canhoto com os dados embaixo.
  return (
    <article className="mx-auto max-w-md overflow-hidden rounded-2xl border bg-card shadow-
lg shadow-primary/10">
      {evento && (
        <header className="flex items-center gap-5 bg-[#1d2559] p-6 text-white">
          <DataDoEvento data={evento.data} />
          <div className="grid gap-1">
            <p className="text-sm opacity-75">Congresso StartUB 2026</p>
            <h1 className="text-2xl leading-tight font-bold">{evento.titulo}</h1>
            <p className="text-sm opacity-75">
              {partesDaData(evento.data).hora}, {evento.local}
            </p>
          </div>
        </header>
      )}

      {/* a linha picotada entre o evento e o canhoto */}
      <div className="relative border-t-2 border-dashed">
        <span className="absolute -top-3 -left-3 size-6 rounded-full bg-background" />
        <span className="absolute -top-3 -right-3 size-6 rounded-full bg-background" />
      </div>

      <div className="grid gap-5 p-6">
```

```
        <div className="flex items-center justify-between gap-3">
          <StatusBadge status={inscricao.status} />
          <span className="text-sm text-muted-foreground">Inscrição nº {inscricao.id}</span>
        </div>

        <dl className="grid gap-3">
          <div>
            <dt className="text-sm text-muted-foreground">Participante</dt>
            <dd className="font-semibold">{inscricao.nome}</dd>
          </div>
          <div>
            <dt className="text-sm text-muted-foreground">E-mail</dt>
            <dd className="font-semibold">{inscricao.email}</dd>
          </div>
          {evento && (
            <div>
              <dt className="text-sm text-muted-foreground">Valor</dt>
              <dd className="font-semibold">{formatarValor(evento.valor)}</dd>
            </div>
          )}
        </dl>

        {inscricao.status === "pendente" && inscricao.link_pagamento && (
          <div className="grid gap-3">
            <Button asChild size="lg">
              <a href={inscricao.link_pagamento} target="_blank" rel="noreferrer">
                Pagar inscrição <ExternalLink />
              </a>
            </Button>
            <p className="flex items-center gap-2 text-sm text-muted-foreground" aria-
live="polite">
              <LoaderCircle className="size-4 animate-spin motion-reduce:animate-none" />
              Esta página atualiza sozinha quando o pagamento for confirmado.
            </p>
          </div>
        )}

        {inscricao.status === "confirmada" && (
          <Button variant="outline" asChild>
            <Link to="/minhas">Ver minhas inscrições</Link>
          </Button>
        )}
      </div>
    </article>
  )
}
```

O comprovante é desenhado como a **credencial do congresso**: o evento numa faixa azul-marinho, com o mesmo `DataDoEvento` da programação, e embaixo o canhoto com os dados da inscrição. Os dois círculos com a cor do fundo, nas pontas da linha tracejada, fazem o recorte do picote. O link de pagamento é um `<a>` com `target="_blank"`, e não um `Link`: ele leva para fora da aplicação. E se alguém abrir o endereço do comprovante de outra pessoa, a API responde 403, e o `MensagemErro` mostra o aviso já traduzido: **Você não tem permissão para isso**.

4.2 Um segundo Context

A API não tem um endereço "minhas inscrições", então a aplicação guarda os **ids** das inscrições feitas neste navegador, no `localStorage`, **separados por usuário**: no laboratório, várias pessoas usam o mesmo computador. O formulário precisa **acrescentar** um id, e a página das minhas inscrições precisa **ler** a lista. É o mesmo problema do usuário logado no Dia 1, e a mesma solução: um Context.

PASSO 4.3 · O CONTEXTO DAS MINHAS INSCRIÇÕES

Crie `src/contexts/MinhasInscricoesContext.tsx`:

src/contexts/MinhasInscricoesContext.tsx

```tsx
import { createContext, useContext, useEffect, useState, type ReactNode } from "react"
import { useAuth } from "@/contexts/AuthContext"

// Guarda só os ids das inscrições feitas neste navegador, separados por usuário:
// no laboratório, várias pessoas usam o mesmo computador.
// Os dados de cada inscrição vêm sempre da API.

const CHAVE = "startub:minhas-inscricoes"

type IdsPorUsuario = Record<string, number[]>

interface MinhasInscricoesValor {
  ids: number[]
  adicionar: (id: number) => void
}

const MinhasInscricoesContext = createContext<MinhasInscricoesValor | null>(null)

function salvos(): IdsPorUsuario {
  try {
    const lido = JSON.parse(localStorage.getItem(CHAVE) ?? "{}")
    return lido && typeof lido === "object" && !Array.isArray(lido) ? lido : {}
  } catch {
    return {}
  }
}

export function MinhasInscricoesProvider({ children }: { children: ReactNode }) {
  const { usuario } = useAuth()
  const [todos, setTodos] = useState<IdsPorUsuario>(salvos)

  useEffect(() => {
    localStorage.setItem(CHAVE, JSON.stringify(todos))
  }, [todos])

  const dono = String(usuario?.id ?? "")
  const lidos = todos[dono]
  const ids = Array.isArray(lidos) ? lidos.filter(Number.isInteger) : []

  function adicionar(id: number) {
    setTodos((atual) => {
      const meus = atual[dono] ?? []
```

```
      return meus.includes(id) ? atual : { ...atual, [dono]: [id, ...meus] }
    })
  }

  return (
    <MinhasInscricoesContext.Provider value={{ ids, adicionar }}>
      {children}
    </MinhasInscricoesContext.Provider>
  )
}

export function useMinhasInscricoes() {
  const valor = useContext(MinhasInscricoesContext)
  if (!valor) throw new Error("useMinhasInscricoes precisa estar dentro de
<MinhasInscricoesProvider>")
  return valor
}
```

Um contexto pode usar outro: o `MinhasInscricoesProvider` chama `useAuth()` para saber de quem são os ids. Por isso, no `main.tsx`, ele precisa ficar **dentro** do `AuthProvider`.

> **Atenção.** O `localStorage` guarda texto, e o que está lá pode ter sido apagado, editado à mão ou gravado por uma versão antiga da aplicação. Por isso o `salvos` tem `try/catch` e confere o formato, e os ids passam por `Number.isInteger`. Sem isso, um único valor estragado derruba a aplicação na abertura.

PASSO 4.4 · O FORMULÁRIO LEVA AO COMPROVANTE

Atualize `src/components/FormularioInscricao.tsx`. O formulário guarda o id no contexto e navega para o comprovante, em vez de limpar os campos:

`src/components/FormularioInscricao.tsx`

```
import { useNavigate } from "react-router"
import { useForm } from "react-hook-form"
import { zodResolver } from "@hookform/resolvers/zod"
import { useMutation } from "@tanstack/react-query"
import { toast } from "sonner"
import { Button } from "@/components/ui/button"
import { Card, CardContent, CardDescription, CardHeader, CardTitle } from
"@/components/ui/card"
import { Field, FieldError, FieldGroup, FieldLabel } from "@/components/ui/field"
import { Input } from "@/components/ui/input"
import { useAuth } from "@/contexts/AuthContext"
import { useMinhasInscricoes } from "@/contexts/MinhasInscricoesContext"
import { aplicarErrosDaApi, traduzir } from "@/lib/erros"
import { ehGratuito, formatarValor } from "@/lib/formatar"
import { esquemaInscricao, type DadosInscricao } from "@/schemas/inscricao"
import { inscrever } from "@/services/inscricoes"
import type { Evento } from "@/tipos"

export function FormularioInscricao({ evento }: { evento: Evento }) {
  const navigate = useNavigate()
  const { usuario } = useAuth()
```

```
  const { adicionar } = useMinhasInscricoes()

  const {
    register,
    handleSubmit,
    setError,
    formState: { errors },
  } = useForm<DadosInscricao>({
    resolver: zodResolver(esquemaInscricao),
    // nome e e-mail já vêm da conta; a pessoa só confere e completa o CPF
    defaultValues: { nome: usuario?.name ?? "", email: usuario?.email ?? "", cpf: "" },
  })

  const inscricao = useMutation({
    mutationFn: (dados: DadosInscricao) => inscrever(evento.id, dados),
    onSuccess: (nova) => {
      adicionar(nova.id)
      toast.success("Inscrição realizada!")
      navigate(`/inscricoes/${nova.id}`)
    },
    onError: (erro) => {
      // 422: erros de campo. Qualquer outro (409, 502, rede): aviso geral.
      if (!aplicarErrosDaApi(erro, setError)) toast.error(traduzir(erro.message))
    },
  })

  return (
    <Card className="h-fit lg:sticky lg:top-24">
      <CardHeader>
        <CardTitle className="text-2xl font-bold">Sua inscrição</CardTitle>
        <CardDescription>
          {ehGratuito(evento.valor)
            ? "Inscrição gratuita. Confira seus dados e confirme."
            : `Inscrição de ${formatarValor(evento.valor)}. O pagamento abre depois que você
confirmar.`}
        </CardDescription>
      </CardHeader>
      <CardContent>
        <form onSubmit={handleSubmit((dados) => inscricao.mutate(dados))} noValidate>
          <FieldGroup>
            <Field data-invalid={!!errors.nome}>
              <FieldLabel htmlFor="nome">Nome completo</FieldLabel>
              <Input id="nome" aria-invalid={!!errors.nome} {...register("nome")} />
              <FieldError errors={[errors.nome]} />
            </Field>

            <Field data-invalid={!!errors.email}>
              <FieldLabel htmlFor="email">E-mail</FieldLabel>
              <Input id="email" type="email" aria-invalid={!!errors.email}
{...register("email")} />
              <FieldError errors={[errors.email]} />
            </Field>

            <Field data-invalid={!!errors.cpf}>
              <FieldLabel htmlFor="cpf">CPF</FieldLabel>
              <Input
```

```
                id="cpf"
                inputMode="numeric"
                maxLength={11}
                placeholder="Somente números"
                aria-invalid={!!errors.cpf}
                {...register("cpf")}
              />
              <FieldError errors={[errors.cpf]} />
            </Field>

            <Button type="submit" disabled={inscricao.isPending}>
              {inscricao.isPending ? "Enviando..." : "Confirmar inscrição"}
            </Button>
          </FieldGroup>
        </form>
      </CardContent>
    </Card>
  )
}
```

PASSO 4.5 · A PÁGINA DAS MINHAS INSCRIÇÕES

Crie `src/pages/MinhasInscricoes.tsx`:

src/pages/MinhasInscricoes.tsx

```tsx
import { Link } from "react-router"
import { useQueries } from "@tanstack/react-query"
import { DataDoEvento } from "@/components/DataDoEvento"
import { Vazio } from "@/components/Estados"
import { StatusBadge } from "@/components/StatusBadge"
import { Card } from "@/components/ui/card"
import { Skeleton } from "@/components/ui/skeleton"
import { useMinhasInscricoes } from "@/contexts/MinhasInscricoesContext"
import { buscarInscricao } from "@/services/inscricoes"

export function MinhasInscricoes() {
  const { ids } = useMinhasInscricoes()

  // Uma consulta por inscrição, todas em paralelo. A chave é a mesma do
  // comprovante, então o que já foi buscado lá aparece aqui sem nova requisição.
  const consultas = useQueries({
    queries: ids.map((id) => ({
      queryKey: ["inscricoes", id],
      queryFn: () => buscarInscricao(id),
    })),
  })

  return (
    <section className="grid gap-6">
      <h1 className="text-4xl font-extrabold sm:text-5xl">Minhas inscrições</h1>

      {ids.length === 0 && (
        <Vazio>
          Você ainda não se inscreveu em nenhum evento.{" "}
          <Link to="/" className="font-semibold text-primary underline">Ver a
```

```
programação</Link>
        </Vazio>
      )}

      <div className="grid gap-3">
        {consultas.map(({ data: inscricao, isPending }, i) =>
          isPending || !inscricao ? (
            <Skeleton key={ids[i]} className="h-28 rounded-xl" />
          ) : (
            <Link key={inscricao.id} to={`/inscricoes/${inscricao.id}`} className="rounded-
xl">
              <Card className="grid grid-cols-[4.5rem_1fr] items-center gap-4 px-5 text-base
hover:bg-muted/60 sm:grid-cols-[4.5rem_1fr_auto]">
                {inscricao.evento && <DataDoEvento data={inscricao.evento.data} />}
                <div>
                  <h2 className="text-lg leading-tight font-bold">{inscricao.evento?.titulo}
</h2>
                  <p className="text-sm text-muted-foreground">Inscrição nº {inscricao.id}
</p>
                </div>
                <StatusBadge status={inscricao.status} />
              </Card>
            </Link>
          ),
        )}
      </div>
    </section>
  )
}
```

Cada inscrição vira uma linha com o `DataDoEvento` à esquerda, o mesmo da programação: um componente bem feito no Dia 1 é reaproveitado sem mudar uma linha. O `useQueries` faz uma consulta por id, todas em paralelo. A chave `["inscricoes", id]` é a mesma do comprovante: o que já foi buscado lá aparece aqui de imediato.

PASSO 4.6 · ROTAS, MENU E PROVIDER

Atualize `src/App.tsx`, com as duas rotas novas dentro da rota protegida:

src/App.tsx

```tsx
import { Route, Routes } from "react-router"
import { Layout } from "@/components/Layout"
import { RotaProtegida } from "@/components/RotaProtegida"
import { Cadastro } from "@/pages/Cadastro"
import { Comprovante } from "@/pages/Comprovante"
import { DetalheEvento } from "@/pages/DetalheEvento"
import { Entrar } from "@/pages/Entrar"
import { MinhasInscricoes } from "@/pages/MinhasInscricoes"
import { NaoEncontrada } from "@/pages/NaoEncontrada"
import { Vitrine } from "@/pages/Vitrine"

export default function App() {
  return (
    <Routes>
      <Route element={<Layout />}>
```

```
        {/* abertas: quem ainda não entrou */}
        <Route path="entrar" element={<Entrar />} />
        <Route path="cadastro" element={<Cadastro />} />

        {/* todo o resto exige login, porque a API exige token */}
        <Route element={<RotaProtegida />}>
          <Route index element={<Vitrine />} />
          <Route path="eventos/:id" element={<DetalheEvento />} />
          <Route path="inscricoes/:id" element={<Comprovante />} />
          <Route path="minhas" element={<MinhasInscricoes />} />
        </Route>

        <Route path="*" element={<NaoEncontrada />} />
      </Route>
    </Routes>
  )
}
```

`src/components/Layout.tsx`, com o item novo no menu:

src/components/Layout.tsx

```
import type { ReactNode } from "react"
import { NavLink, Outlet } from "react-router"
import { LogOut } from "lucide-react"
import { Button } from "@/components/ui/button"
import { useAuth } from "@/contexts/AuthContext"
import { cn } from "@/lib/utils"

function Item({ para, children }: { para: string; children: ReactNode }) {
  return (
    <NavLink
      to={para}
      end
      className={({ isActive }) =>
        cn(
          "border-b-2 py-1 text-sm font-semibold",
          isActive ? "border-primary text-foreground" : "border-transparent text-muted-
foreground hover:text-foreground",
        )
      }
    >
      {children}
    </NavLink>
  )
}

export function Layout() {
  const { usuario, sair } = useAuth()

  return (
    <div className="flex min-h-svh flex-col">
      <header className="sticky top-0 z-10 border-b bg-background/90 backdrop-blur">
        <div className="mx-auto flex max-w-5xl flex-wrap items-center gap-x-6 gap-y-2 px-4
py-3">
          <NavLink to="/" className="mr-auto">
```

```
            <img src="/startub-logo.jpg" alt="Congresso StartUB 2026" className="h-10 w-auto
rounded-md dark:bg-white dark:p-1" />
          </NavLink>
          <nav className="flex flex-wrap items-center gap-x-5 gap-y-1">
            {usuario ? (
              <>
                <Item para="/">Programação</Item>
                <Item para="/minhas">Minhas inscrições</Item>
                <Button variant="ghost" size="sm" onClick={sair} title={`Sair da conta de
${usuario.name}`}>
                  <LogOut /> <span className="sr-only sm:not-sr-only">Sair</span>
                </Button>
              </>
            ) : (
              <>
                <Item para="/entrar">Entrar</Item>
                <Item para="/cadastro">Criar conta</Item>
              </>
            )}
          </nav>
        </div>
      </header>

      <main className="mx-auto w-full max-w-5xl flex-1 px-4 py-10">
        <Outlet />
      </main>

      <footer className="onda-startub mt-10 pb-8 text-center text-sm">
        <p className="font-semibold">II Congresso de Ciência, Empreendedorismo e Inovação da
Unibalsas</p>
        <p className="opacity-75">Feito no minicurso de frontend com React e TypeScript.</p>
      </footer>
    </div>
  )
}
```

E `src/main.tsx`, com o provider novo dentro do `AuthProvider`:

src/main.tsx

```
import { StrictMode } from "react"
import { createRoot } from "react-dom/client"
import { BrowserRouter } from "react-router"
import { QueryClient, QueryClientProvider } from "@tanstack/react-query"
import { ReactQueryDevtools } from "@tanstack/react-query-devtools"

import "./index.css"
import App from "./App.tsx"
import { ThemeProvider } from "@/components/theme-provider.tsx"
import { Toaster } from "@/components/ui/sonner"
import { AuthProvider } from "@/contexts/AuthContext"
import { MinhasInscricoesProvider } from "@/contexts/MinhasInscricoesContext"

const queryClient = new QueryClient({
  defaultOptions: {
    queries: { staleTime: 30_000 }, // dados valem 30 s antes de buscar de novo
```

```
  },
})

createRoot(document.getElementById("root")!).render(
  <StrictMode>
    <ThemeProvider>
      <QueryClientProvider client={queryClient}>
        <BrowserRouter>
          <AuthProvider>
            <MinhasInscricoesProvider>
              <App />
            </MinhasInscricoesProvider>
          </AuthProvider>
        </BrowserRouter>
        <Toaster richColors position="top-center" />
        <ReactQueryDevtools />
      </QueryClientProvider>
    </ThemeProvider>
  </StrictMode>
)
```

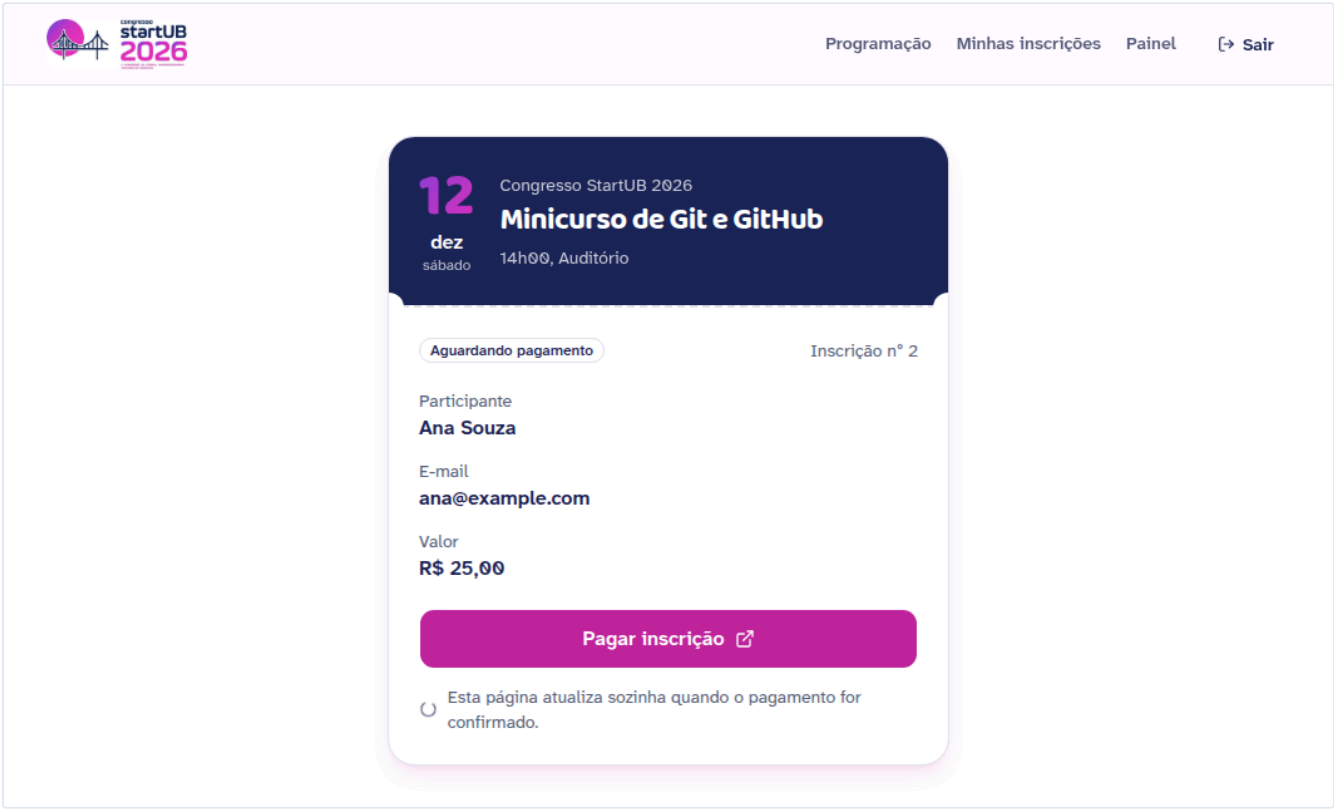


Figura 6. O comprovante de um evento pago, esperando a confirmação

CONFIRA ANTES DE SEGUIR. A inscrição num evento pago leva ao comprovante com **Aguardando pagamento** e o botão **Pagar inscrição**, que abre a página de pagamento do Asaas sandbox. Com o pagamento confirmado no sandbox, o que o professor faz pelo painel do Asaas durante o encontro, a página muda para **Confirmada** sozinha em até 5 segundos, e a aba Network mostra as consultas de 5 em 5 segundos pararem. **Minhas inscrições** lista o que você fez, continua lá depois de recarregar e fica vazia se outra pessoa entrar no mesmo navegador.

5. O painel do organizador

5.1 Um formulário com números e datas

Toda conta pode criar eventos, e todo evento criado ganha um dono: o `user_id` da conta que o criou. A API deixa **qualquer conta ver** os eventos, porque é a vitrine, mas só o **dono altera ou remove**. Para os outros, a resposta é 403. Os eventos do StartUB não têm dono, então ninguém os altera pela aplicação:

PATCH	https://startub.appmake.com.br/api/eventos/1	403 Forbidden

corpo enviado

```
{
  "vagas": 30
}
```

RESPOSTA

```
{
  "message": "This action is unauthorized."
}
```

A mesma regra vale para as inscrições: só quem se inscreveu consulta o próprio comprovante, e só o dono de um evento vê a lista de inscritos dele. Por isso o painel mostra **só os eventos que a conta criou**.

O formulário de evento tem uma dificuldade nova: o que um `<input>` entrega é sempre **texto**, mas a API espera `vagas` e `valor` como **números**. O Zod resolve com `z.coerce.number()`, que converte antes de validar. Com isso, o formulário passa a ter dois tipos: o que ele **guarda** (texto) e o que ele **entrega** depois de validar (números).

PASSO 5.1 · O ESQUEMA DO EVENTO

Crie `src/schemas/evento.ts`:

src/schemas/evento.ts

```
import { z } from "zod"

// O campo <input type="datetime-local"> entrega "2026-12-05T19:00".
// O Laravel aceita esse formato e devolve "2026-12-05 19:00".
export const esquemaEvento = z.object({
  titulo: z.string().trim().min(1, "Campo obrigatório.").max(120, "Use no máximo 120
caracteres."),
  descricao: z.string().trim(),
  local: z.string().trim().min(1, "Campo obrigatório.").max(120, "Use no máximo 120
caracteres."),
  data: z
    .string()
    .min(1, "Campo obrigatório.")
    .refine((texto) => new Date(texto) > new Date(), "A data precisa ser futura."),
  vagas: z.coerce.number<string>().int("Use um número inteiro.").min(1, "O mínimo é 1
vaga."),
```

```
  valor: z.coerce.number<string>().min(0, "O valor não pode ser negativo."),
})

// O que o formulário guarda (texto) e o que a API recebe (números).
export type CamposEvento = z.input<typeof esquemaEvento>
export type DadosEvento = z.output<typeof esquemaEvento>
```

O `refine` cria uma regra que o Zod não tem pronta, como "a data precisa ser futura", a mesma `after:now` da API. O `z.input` e o `z.output` extraem os dois tipos do mesmo esquema.

PASSO 5.2 · CRIAR, ALTERAR E REMOVER

Atualize `src/services/eventos.ts`:

src/services/eventos.ts

```
import type { Evento } from "@/tipos"
import type { DadosEvento } from "@/schemas/evento"
import { requisicao } from "./api"

export function listarEventos() {
  return requisicao<Evento[]>("/eventos")
}

export function buscarEvento(id: number) {
  return requisicao<Evento>(`/eventos/${id}`)
}

export function criarEvento(dados: DadosEvento) {
  return requisicao<Evento>("/eventos", {
    method: "POST",
    body: JSON.stringify(dados),
  })
}

export function atualizarEvento(id: number, dados: DadosEvento) {
  return requisicao<Evento>(`/eventos/${id}`, {
    method: "PATCH",
    body: JSON.stringify(dados),
  })
}

export function removerEvento(id: number) {
  return requisicao<void>(`/eventos/${id}`, { method: "DELETE" })
}
```

PASSO 5.3 · O FORMULÁRIO DE EVENTO

Crie `src/components/FormularioEvento.tsx`:

src/components/FormularioEvento.tsx

```
import { useForm } from "react-hook-form"
import { zodResolver } from "@hookform/resolvers/zod"
import { useMutation, useQueryClient } from "@tanstack/react-query"
```

```tsx
import { toast } from "sonner"
import { Button } from "@/components/ui/button"
import { Field, FieldError, FieldGroup, FieldLabel } from "@/components/ui/field"
import { Input } from "@/components/ui/input"
import { Textarea } from "@/components/ui/textarea"
import { aplicarErrosDaApi, traduzir } from "@/lib/erros"
import { esquemaEvento, type CamposEvento, type DadosEvento } from "@/schemas/evento"
import { atualizarEvento, criarEvento } from "@/services/eventos"
import type { Evento } from "@/tipos"

interface FormularioEventoProps {
  evento?: Evento // sem evento: criar; com evento: editar
  aoConcluir: () => void
}

// Converte o evento da API para os campos do formulário (tudo texto).
function paraCampos(evento?: Evento): CamposEvento {
  return {
    titulo: evento?.titulo ?? "",
    descricao: evento?.descricao ?? "",
    local: evento?.local ?? "",
    data: evento ? evento.data.replace(" ", "T") : "",
    vagas: evento ? String(evento.vagas) : "",
    valor: evento ? evento.valor : "0",
  }
}

export function FormularioEvento({ evento, aoConcluir }: FormularioEventoProps) {
  const queryClient = useQueryClient()

  const {
    register,
    handleSubmit,
    setError,
    formState: { errors },
  } = useForm<CamposEvento, unknown, DadosEvento>({
    resolver: zodResolver(esquemaEvento),
    defaultValues: paraCampos(evento),
  })

  const salvar = useMutation({
    mutationFn: (dados: DadosEvento) =>
      evento ? atualizarEvento(evento.id, dados) : criarEvento(dados),
    onSuccess: () => {
      // A lista mudou: tudo que começa com ["eventos"] será buscado de novo.
      queryClient.invalidateQueries({ queryKey: ["eventos"] })
      toast.success(evento ? "Evento atualizado." : "Evento criado.")
      aoConcluir()
    },
    onError: (erro) => {
      if (!aplicarErrosDaApi(erro, setError)) toast.error(traduzir(erro.message))
    },
  })

  return (
    <form onSubmit={handleSubmit((dados) => salvar.mutate(dados))} noValidate>
```

```
      <FieldGroup>
        <Field data-invalid={!!errors.titulo}>
          <FieldLabel htmlFor="titulo">Título</FieldLabel>
          <Input id="titulo" aria-invalid={!!errors.titulo} {...register("titulo")} />
          <FieldError errors={[errors.titulo]} />
        </Field>

        <Field data-invalid={!!errors.descricao}>
          <FieldLabel htmlFor="descricao">Descrição</FieldLabel>
          <Textarea id="descricao" rows={3} {...register("descricao")} />
          <FieldError errors={[errors.descricao]} />
        </Field>

        <div className="grid gap-4 sm:grid-cols-2">
          <Field data-invalid={!!errors.local}>
            <FieldLabel htmlFor="local">Local</FieldLabel>
            <Input id="local" aria-invalid={!!errors.local} {...register("local")} />
            <FieldError errors={[errors.local]} />
          </Field>
          <Field data-invalid={!!errors.data}>
            <FieldLabel htmlFor="data">Data e hora</FieldLabel>
            <Input id="data" type="datetime-local" aria-invalid={!!errors.data}
{...register("data")} />
            <FieldError errors={[errors.data]} />
          </Field>
          <Field data-invalid={!!errors.vagas}>
            <FieldLabel htmlFor="vagas">Vagas</FieldLabel>
            <Input id="vagas" type="number" min={1} aria-invalid={!!errors.vagas}
{...register("vagas")} />
            <FieldError errors={[errors.vagas]} />
          </Field>
          <Field data-invalid={!!errors.valor}>
            <FieldLabel htmlFor="valor">Valor (R$)</FieldLabel>
            <Input id="valor" type="number" min={0} step="0.01" aria-invalid=
{!!errors.valor} {...register("valor")} />
            <FieldError errors={[errors.valor]} />
          </Field>
        </div>

        <Button type="submit" disabled={salvar.isPending}>
          {salvar.isPending ? "Salvando..." : "Salvar evento"}
        </Button>
      </FieldGroup>
    </form>
  )
}
```

O mesmo componente cria e edita. Sem `evento`, os campos começam vazios e o envio faz `POST`. Com `evento`, a função `paraCampos` converte o que veio da API para texto, inclusive a data, de `"2026-12-05 19:00"` para o `"2026-12-05T19:00"` que o campo `datetime-local` exige, e o envio faz `PATCH`.

Conceito. O `invalidateQueries({ queryKey: ["eventos"] })` avisa o cache de que tudo que começa com `["eventos"]` ficou velho. A lista **e** cada evento aberto são buscados de novo na próxima vez que aparecerem. É isso que faz a vitrine mostrar o evento novo sem ninguém recarregar a página.

PASSO 5.4 · O PAINEL

Crie `src/pages/Painel.tsx`:

src/pages/Painel.tsx

```tsx
import { useState } from "react"
import { useMutation, useQuery, useQueryClient } from "@tanstack/react-query"
import { Pencil, Plus, Trash2 } from "lucide-react"
import { toast } from "sonner"
import { FormularioEvento } from "@/components/FormularioEvento"
import { MensagemErro, Vazio } from "@/components/Estados"
import {
  AlertDialog,
  AlertDialogAction,
  AlertDialogCancel,
  AlertDialogContent,
  AlertDialogDescription,
  AlertDialogFooter,
  AlertDialogHeader,
  AlertDialogTitle,
  AlertDialogTrigger,
} from "@/components/ui/alert-dialog"
import { Button } from "@/components/ui/button"
import { Dialog, DialogContent, DialogHeader, DialogTitle } from "@/components/ui/dialog"
import { Skeleton } from "@/components/ui/skeleton"
import { useAuth } from "@/contexts/AuthContext"
import { traduzir } from "@/lib/erros"
import { formatarData, formatarValor } from "@/lib/formatar"
import { listarEventos, removerEvento } from "@/services/eventos"
import type { Evento } from "@/tipos"

// null: diálogo fechado · "novo": criando · Evento: editando aquele evento
type Edicao = null | "novo" | Evento

export function Painel() {
  const { usuario } = useAuth()
  const queryClient = useQueryClient()
  const [edicao, setEdicao] = useState<Edicao>(null)

  // Mesma chave da vitrine: os dois compartilham o cache.
  const { data: eventos, isPending, error } = useQuery({
    queryKey: ["eventos"],
    queryFn: listarEventos,
  })

  // A API só deixa alterar e remover o que a própria conta criou.
  // O painel mostra só esses: estado derivado da mesma lista da vitrine.
  const meus = eventos?.filter((evento) => evento.user_id === usuario?.id)
```

```
  const remover = useMutation({
    mutationFn: removerEvento,
    onSuccess: () => {
      queryClient.invalidateQueries({ queryKey: ["eventos"] })
      toast.success("Evento removido.")
    },
    onError: (erro) => toast.error(traduzir(erro.message)),
  })

  return (
    <section className="grid gap-6">
      <div className="flex flex-wrap items-end justify-between gap-2">
        <div>
          <h1 className="text-4xl font-extrabold sm:text-5xl">Painel do organizador</h1>
          <p className="text-muted-foreground">Olá, {usuario?.name}. Aqui ficam os eventos
que você criou.</p>
        </div>
        <Button onClick={() => setEdicao("novo")}>
          <Plus /> Novo evento
        </Button>
      </div>

      {isPending && <Skeleton className="h-64 rounded-xl" />}
      {error && <MensagemErro erro={error} />}
      {meus?.length === 0 && <Vazio>Você ainda não criou nenhum evento.</Vazio>}

      {meus && meus.length > 0 && (
        <ul className="divide-y rounded-xl border">
          {meus.map((evento) => (
            <li key={evento.id} className="flex flex-wrap items-center gap-3 p-4">
              <div className="mr-auto">
                <p className="font-medium">{evento.titulo}</p>
                <p className="text-sm text-muted-foreground">
                  {formatarData(evento.data)}, {evento.vagas} vagas,
{formatarValor(evento.valor)}
                </p>
              </div>
              <Button variant="outline" size="sm" onClick={() => setEdicao(evento)}>
                <Pencil /> Editar
              </Button>
              <AlertDialog>
                <AlertDialogTrigger asChild>
                  <Button variant="destructive" size="sm">
                    <Trash2 /> Remover
                  </Button>
                </AlertDialogTrigger>
                <AlertDialogContent>
                  <AlertDialogHeader>
                    <AlertDialogTitle>Remover "{evento.titulo}"?</AlertDialogTitle>
                    <AlertDialogDescription>
                      As inscrições deste evento também serão apagadas. Não dá para
desfazer.
                    </AlertDialogDescription>
                  </AlertDialogHeader>
                  <AlertDialogFooter>
                    <AlertDialogCancel>Cancelar</AlertDialogCancel>
```

```
                    <AlertDialogAction onClick={() =>
remover.mutate(evento.id)}>Remover</AlertDialogAction>
                  </AlertDialogFooter>
                </AlertDialogContent>
              </AlertDialog>
            </li>
          ))}
        </ul>
      )}

      <Dialog open={edicao !== null} onOpenChange={(aberto) => !aberto && setEdicao(null)}>
        <DialogContent className="sm:max-w-xl">
          <DialogHeader>
            <DialogTitle>{edicao === "novo" ? "Novo evento" : "Editar evento"}</DialogTitle>
          </DialogHeader>
          {edicao !== null && (
            <FormularioEvento
              key={edicao === "novo" ? "novo" : edicao.id}
              evento={edicao === "novo" ? undefined : edicao}
              aoConcluir={() => setEdicao(null)}
            />
          )}
        </DialogContent>
      </Dialog>
    </section>
  )
}
```

A lista do painel é **estado derivado**, como a busca da vitrine: a mesma consulta `["eventos"]`, filtrada por `evento.user_id === usuario?.id`. Nenhuma requisição a mais, e o painel nunca oferece um botão que a API recusaria. O estado `edicao` diz o que o diálogo mostra: `null` fechado, `"novo"` para criar, ou o evento que está sendo editado. A `key` no `FormularioEvento` faz o React criar um formulário novo a cada evento aberto, com os valores certos. A remoção passa pelo `AlertDialog`, que pede confirmação.

PASSO 5.5 · ROTA E MENU

Atualize `src/App.tsx`, com a rota do painel dentro da rota protegida:

src/App.tsx

```
import { Route, Routes } from "react-router"
import { Layout } from "@/components/Layout"
import { RotaProtegida } from "@/components/RotaProtegida"
import { Cadastro } from "@/pages/Cadastro"
import { Comprovante } from "@/pages/Comprovante"
import { DetalheEvento } from "@/pages/DetalheEvento"
import { Entrar } from "@/pages/Entrar"
import { MinhasInscricoes } from "@/pages/MinhasInscricoes"
import { NaoEncontrada } from "@/pages/NaoEncontrada"
import { Painel } from "@/pages/Painel"
import { Vitrine } from "@/pages/Vitrine"

export default function App() {
  return (
```

```
    <Routes>
      <Route element={<Layout />}>
        {/* abertas: quem ainda não entrou */}
        <Route path="entrar" element={<Entrar />} />
        <Route path="cadastro" element={<Cadastro />} />

        {/* todo o resto exige login, porque a API exige token */}
        <Route element={<RotaProtegida />}>
          <Route index element={<Vitrine />} />
          <Route path="eventos/:id" element={<DetalheEvento />} />
          <Route path="inscricoes/:id" element={<Comprovante />} />
          <Route path="minhas" element={<MinhasInscricoes />} />
          <Route path="painel" element={<Painel />} />
        </Route>

        <Route path="*" element={<NaoEncontrada />} />
      </Route>
    </Routes>
  )
}
```

E `src/components/Layout.tsx`, com o item **Painel**:

src/components/Layout.tsx

```
import type { ReactNode } from "react"
import { NavLink, Outlet } from "react-router"
import { LogOut } from "lucide-react"
import { Button } from "@/components/ui/button"
import { useAuth } from "@/contexts/AuthContext"
import { cn } from "@/lib/utils"

function Item({ para, children }: { para: string; children: ReactNode }) {
  return (
    <NavLink
      to={para}
      end
      className={({ isActive }) =>
        cn(
          "border-b-2 py-1 text-sm font-semibold",
          isActive ? "border-primary text-foreground" : "border-transparent text-muted-
foreground hover:text-foreground",
        )
      }
    >
      {children}
    </NavLink>
  )
}

export function Layout() {
  const { usuario, sair } = useAuth()

  return (
    <div className="flex min-h-svh flex-col">
      <header className="sticky top-0 z-10 border-b bg-background/90 backdrop-blur">
```

```
        <div className="mx-auto flex max-w-5xl flex-wrap items-center gap-x-6 gap-y-2 px-4
py-3">
          <NavLink to="/" className="mr-auto">
            <img src="/startub-logo.jpg" alt="Congresso StartUB 2026" className="h-10 w-auto
rounded-md dark:bg-white dark:p-1" />
          </NavLink>
          <nav className="flex flex-wrap items-center gap-x-5 gap-y-1">
            {usuario ? (
              <>
                <Item para="/">Programação</Item>
                <Item para="/minhas">Minhas inscrições</Item>
                <Item para="/painel">Painel</Item>
                <Button variant="ghost" size="sm" onClick={sair} title={`Sair da conta de
${usuario.name}`}>
                  <LogOut /> <span className="sr-only sm:not-sr-only">Sair</span>
                </Button>
              </>
            ) : (
              <>
                <Item para="/entrar">Entrar</Item>
                <Item para="/cadastro">Criar conta</Item>
              </>
            )}
          </nav>
        </div>
      </header>

      <main className="mx-auto w-full max-w-5xl flex-1 px-4 py-10">
        <Outlet />
      </main>

      <footer className="onda-startub mt-10 pb-8 text-center text-sm">
        <p className="font-semibold">II Congresso de Ciência, Empreendedorismo e Inovação da
Unibalsas</p>
        <p className="opacity-75">Feito no minicurso de frontend com React e TypeScript.</p>
      </footer>
    </div>
  )
}
```

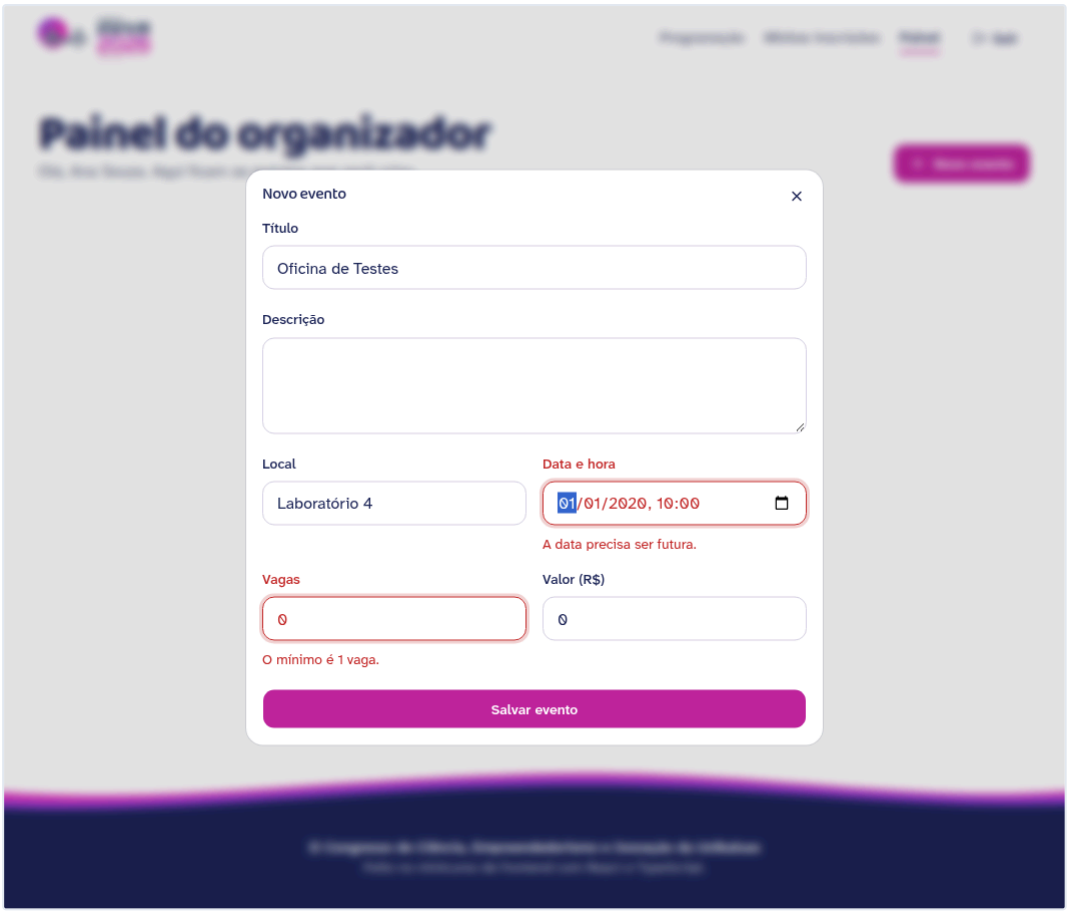


Figura 7. O formulário de evento no painel, com as regras do Zod

> **CONFIRA ANTES DE SEGUIR.** Numa conta nova, o painel mostra **Você ainda não criou nenhum evento.**, mesmo com a vitrine cheia. **Novo evento** com data no passado e vagas zero mostra os dois erros sem chamar a API. Um evento válido aparece no painel e, ao voltar para a vitrine, também lá. **Editar** abre o formulário preenchido, inclusive a data. **Remover** pede confirmação e tira o evento das duas telas.

6. Cadastro e login com Zod

No Dia 1, o cadastro e o login usaram um `useState` por campo e só mostravam os erros que a API devolvia. Agora que o projeto tem React Hook Form e Zod, os dois formulários ficam iguais aos outros: as regras são conferidas antes de enviar, e os erros da API vão para os campos pelo mesmo `aplicarErrosDaApi`.

O cadastro tem uma regra que envolve **dois campos**: a confirmação precisa ser igual à senha. A API recusa assim:

POST	https://startub.appmake.com.br/api/register	422 Unprocessable Content

corpo enviado

```
{
  "name": "Ana Souza",
  "email": "ana.teste@example.com",
  "password": "segredo123",
```

```
  "password_confirmation": "segredo999"
}
```

RESPOSTA

```
{
  "message": "The password field confirmation does not match.",
  "errors": {
    "password": [
      "The password field confirmation does not match."
    ]
  }
}
```

Repare que a API põe o erro no campo `password`. No formulário, faz mais sentido mostrá-lo embaixo da **confirmação**, que é onde a pessoa errou. O Zod permite escolher.

PASSO 6.1 · OS ESQUEMAS

Crie `src/schemas/login.ts`:

`src/schemas/login.ts`

```
import { z } from "zod"

export const esquemaLogin = z.object({
  email: z.email("Informe um e-mail válido."),
  password: z.string().min(1, "Informe a senha."),
})
```

E `src/schemas/cadastro.ts`:

`src/schemas/cadastro.ts`

```
import { z } from "zod"

// As regras do cadastro na API: nome até 255, e-mail válido,
// senha com pelo menos 8 caracteres e confirmação igual à senha.
export const esquemaCadastro = z
  .object({
    name: z.string().trim().min(1, "Campo obrigatório.").max(255, "Use no máximo 255
caracteres."),
    email: z.email("Informe um e-mail válido."),
    password: z.string().min(8, "Use pelo menos 8 caracteres."),
    password_confirmation: z.string(),
  })
  // Regra que envolve dois campos: vai no objeto, e o erro aparece na confirmação.
  .refine((dados) => dados.password === dados.password_confirmation, {
    message: "As senhas não conferem.",
    path: ["password_confirmation"],
  })
```

Uma regra que compara dois campos não cabe em nenhum dos dois: ela vai no **objeto**, com `.refine`, e o `path` diz em qual campo o erro aparece.

PASSO 6.2 · O LOGIN

Troque o conteúdo de `src/pages/Entrar.tsx`:

src/pages/Entrar.tsx

```
import { Link, Navigate, useLocation } from "react-router"
import { useForm } from "react-hook-form"
import { zodResolver } from "@hookform/resolvers/zod"
import { useMutation } from "@tanstack/react-query"
import { toast } from "sonner"
import { TelaDeAcesso } from "@/components/TelaDeAcesso"
import { Button } from "@/components/ui/button"
import { Card, CardContent, CardDescription, CardHeader, CardTitle } from
"@/components/ui/card"
import { Field, FieldError, FieldGroup, FieldLabel } from "@/components/ui/field"
import { Input } from "@/components/ui/input"
import { useAuth } from "@/contexts/AuthContext"
import { aplicarErrosDaApi, traduzir } from "@/lib/erros"
import { esquemaLogin } from "@/schemas/login"
import type { DadosLogin } from "@/services/auth"

export function Entrar() {
  const { usuario, entrar } = useAuth()
  const local = useLocation()
  const destino = local.state?.voltarPara ?? "/"

  const {
    register,
    handleSubmit,
    setError,
    formState: { errors },
  } = useForm<DadosLogin>({
    resolver: zodResolver(esquemaLogin),
    defaultValues: { email: "", password: "" },
  })

  const login = useMutation({
    mutationFn: entrar,
    onError: (erro) => {
      if (!aplicarErrosDaApi(erro, setError)) toast.error(traduzir(erro.message))
    },
  })

  // Logado, não há o que fazer aqui: segue para onde a pessoa queria ir.
  if (usuario) return <Navigate to={destino} replace />

  return (
    <TelaDeAcesso>
      <Card className="w-full max-w-sm justify-self-center">
        <CardHeader>
          <CardTitle className="text-3xl font-bold">Entrar</CardTitle>
          <CardDescription>Entre para ver a programação e fazer suas inscrições.
```

```
</CardDescription>
        </CardHeader>
        <CardContent>
          <form onSubmit={handleSubmit((dados) => login.mutate(dados))} noValidate>
            <FieldGroup>
              <Field data-invalid={!!errors.email}>
                <FieldLabel htmlFor="email">E-mail</FieldLabel>
                <Input id="email" type="email" autoComplete="username" aria-invalid=
{!!errors.email} {...register("email")} />
                <FieldError errors={[errors.email]} />
              </Field>
              <Field data-invalid={!!errors.password}>
                <FieldLabel htmlFor="password">Senha</FieldLabel>
                <Input id="password" type="password" autoComplete="current-password" aria-
invalid={!!errors.password} {...register("password")} />
                <FieldError errors={[errors.password]} />
              </Field>
              <Button type="submit" disabled={login.isPending}>
                {login.isPending ? "Entrando..." : "Entrar"}
              </Button>
              <p className="text-center text-sm text-muted-foreground">
                Ainda não tem conta? <Link to="/cadastro" className="text-primary
underline">Criar conta</Link>
              </p>
            </FieldGroup>
          </form>
        </CardContent>
      </Card>
    </TelaDeAcesso>
  )
}
```

PASSO 6.3 · O CADASTRO

Troque o conteúdo de `src/pages/Cadastro.tsx`:

src/pages/Cadastro.tsx

```
import { Link, Navigate } from "react-router"
import { useForm } from "react-hook-form"
import { zodResolver } from "@hookform/resolvers/zod"
import { useMutation } from "@tanstack/react-query"
import { toast } from "sonner"
import { TelaDeAcesso } from "@/components/TelaDeAcesso"
import { Button } from "@/components/ui/button"
import { Card, CardContent, CardDescription, CardHeader, CardTitle } from
"@/components/ui/card"
import { Field, FieldError, FieldGroup, FieldLabel } from "@/components/ui/field"
import { Input } from "@/components/ui/input"
import { useAuth } from "@/contexts/AuthContext"
import { aplicarErrosDaApi, traduzir } from "@/lib/erros"
import { esquemaCadastro } from "@/schemas/cadastro"
import type { DadosCadastro } from "@/services/auth"

export function Cadastro() {
  const { usuario, cadastrar } = useAuth()
```

```
  const {
    register,
    handleSubmit,
    setError,
    formState: { errors },
  } = useForm<DadosCadastro>({
    resolver: zodResolver(esquemaCadastro),
    defaultValues: { name: "", email: "", password: "", password_confirmation: "" },
  })

  const cadastro = useMutation({
    mutationFn: cadastrar,
    onSuccess: () => toast.success("Conta criada. Bem-vindo!"),
    onError: (erro) => {
      if (!aplicarErrosDaApi(erro, setError)) toast.error(traduzir(erro.message))
    },
  })

  if (usuario) return <Navigate to="/" replace />

  return (
    <TelaDeAcesso>
      <Card className="w-full max-w-sm justify-self-center">
        <CardHeader>
          <CardTitle className="text-3xl font-bold">Criar conta</CardTitle>
          <CardDescription>Crie sua conta para ver a programação do StartUB 2026 e se
inscrever.</CardDescription>
        </CardHeader>
        <CardContent>
          <form onSubmit={handleSubmit((dados) => cadastro.mutate(dados))} noValidate>
            <FieldGroup>
              <Field data-invalid={!!errors.name}>
                <FieldLabel htmlFor="name">Nome</FieldLabel>
                <Input id="name" autoComplete="name" aria-invalid={!!errors.name}
{...register("name")} />
                <FieldError errors={[errors.name]} />
              </Field>
              <Field data-invalid={!!errors.email}>
                <FieldLabel htmlFor="email">E-mail</FieldLabel>
                <Input id="email" type="email" autoComplete="email" aria-invalid=
{!!errors.email} {...register("email")} />
                <FieldError errors={[errors.email]} />
              </Field>
              <Field data-invalid={!!errors.password}>
                <FieldLabel htmlFor="password">Senha</FieldLabel>
                <Input id="password" type="password" autoComplete="new-password" aria-
invalid={!!errors.password} {...register("password")} />
                <FieldError errors={[errors.password]} />
              </Field>
              <Field data-invalid={!!errors.password_confirmation}>
                <FieldLabel htmlFor="password_confirmation">Confirme a senha</FieldLabel>
                <Input id="password_confirmation" type="password" autoComplete="new-
password" aria-invalid={!!errors.password_confirmation}
{...register("password_confirmation")} />
                <FieldError errors={[errors.password_confirmation]} />
```

```
              </Field>
              <Button type="submit" disabled={cadastro.isPending}>
                {cadastro.isPending ? "Criando..." : "Criar conta"}
              </Button>
              <p className="text-center text-sm text-muted-foreground">
                Já tem conta? <Link to="/entrar" className="text-primary
underline">Entrar</Link>
              </p>
            </FieldGroup>
          </form>
        </CardContent>
      </Card>
    </TelaDeAcesso>
  )
}
```

Compare com a versão do Dia 1: sumiram os quatro `useState`, o `try/catch/finally` e a função `erroDe`. O React Hook Form guarda os valores e os erros, o `useMutation` cuida do envio e do `isPending`, e o Zod confere as regras antes de gastar uma requisição.

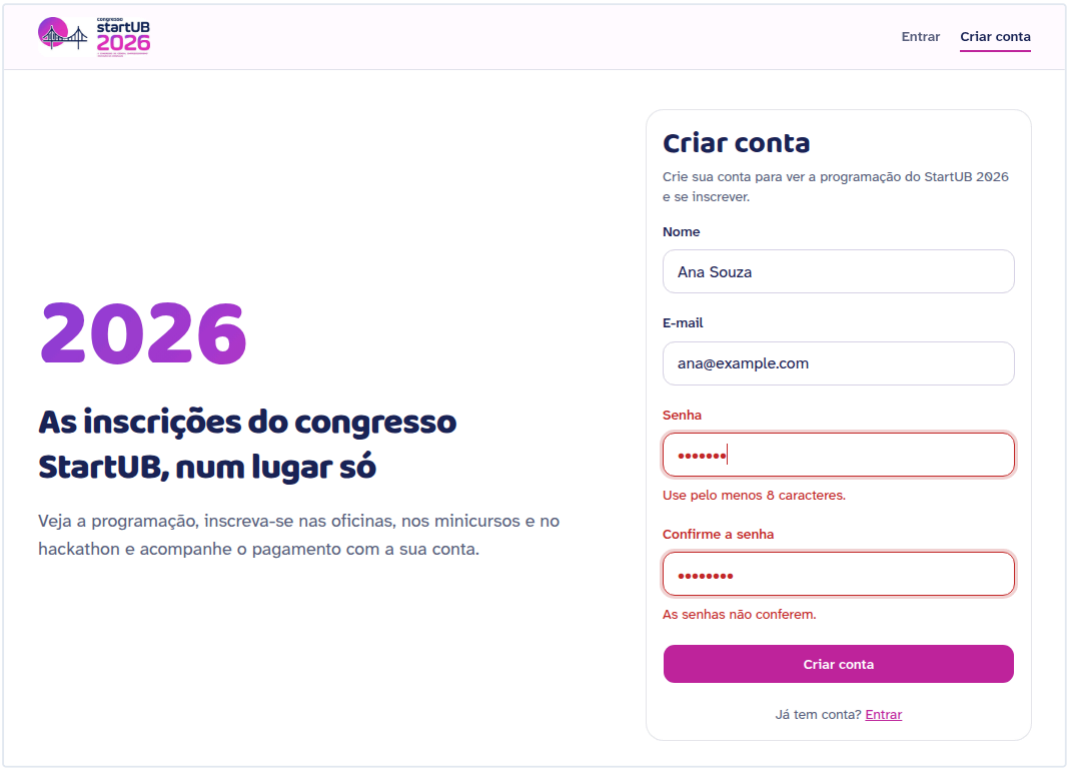


Figura 8. O cadastro com Zod: a confirmação é conferida antes de enviar

> **CONFIRA ANTES DE SEGUIR.** O cadastro com senha de 5 letras mostra **Use pelo menos 8 caracteres.** sem chamar a API. Confirmação diferente mostra **As senhas não conferem.** embaixo da confirmação. Um e-mail já cadastrado mostra **Este e-mail já está em uso.**, agora vindo da API. O login com a senha errada mostra **Credenciais inválidas.** no campo de e-mail.

7. Encerramento

7.1 A versão de produção

O `npm run dev` serve para desenvolver. Para publicar, o Vite gera arquivos otimizados:

```
TERMINAL
> npm run build
> npm run preview
```

O `build` confere os tipos com o TypeScript e gera a pasta `dist`, com um HTML, um CSS e um JavaScript minificados. O `preview` serve essa pasta em `http://localhost:4173`, para conferir antes de publicar.

> **Atenção.** O `build` avisa *Some chunks are larger than 500 kB*. Não é erro: é o Vite sugerindo dividir o código em partes carregadas sob demanda, o que vale a pena em aplicações maiores. Outra: ao publicar numa hospedagem estática, o endereço `/eventos/2` digitado direto no navegador dá 404, porque o servidor procura um arquivo com esse nome. A hospedagem precisa ser configurada para devolver o `index.html` em qualquer endereço.

7.2 Outro caminho: DaisyUI

> **Outro caminho.** O **DaisyUI** é um plugin do Tailwind que entrega componentes como **classes** prontas: `btn`, `card`, `badge`, `input`, e dezenas de temas. Em vez de copiar código de componente para o projeto, como o shadcn, escreve-se `<button className="btn btn-primary">`. É mais rápido para prototipar, por exemplo num hackathon, e dá menos controle sobre o comportamento e a acessibilidade de cada peça. Para experimentar num projeto novo com Tailwind: `npm install -D daisyui` e, no CSS, a linha `@plugin "daisyui";` logo abaixo do `@import "tailwindcss";`. Não misture os dois no mesmo projeto: as classes e os temas entram em conflito.

7.3 Do minicurso às equipes do StartUB

A aplicação que você construiu conversa com a API que a turma do workshop de Laravel construiu, **sem que uma turma precisasse ver o código da outra**. O que as uniu foi o contrato da API, resumido no Anexo A. É assim que equipes de frontend, backend, mobile e design trabalham em paralelo. E é esse o convite do StartUB: formar equipes com gente de áreas diferentes e construir juntos.

> **Fechamento.** Hoje a aplicação passou a escrever na API. O **Zod** descreve as regras e gera o tipo. O **React Hook Form** liga os campos e só envia o que foi aprovado. O `useMutation` envia e trata a resposta, e o erro 422 do servidor volta para o campo certo. O `refetchInterval` acompanha um pagamento que acontece fora da aplicação. Um segundo **Context** guarda as inscrições de cada pessoa. E o `invalidateQueries` mantém todas as telas atualizadas depois de cada alteração. A pergunta que atravessou o encontro, **quem é dono deste dado**, vale para qualquer aplicação que você construir daqui em diante.

Para ir além

Tema	Por onde começar
Atualização otimista	o painel mostra a mudança antes da resposta da API e desfaz se falhar: guia *Optimistic Updates* do TanStack Query
Testes	Vitest e Testing Library, testando os esquemas do Zod e os componentes
Máscara e validação de CPF	formatar enquanto digita e conferir os dígitos verificadores com `refine`
Listar os inscritos no painel	`GET /eventos/{id}/inscricoes`, que só responde ao dono do evento
Next.js	React com renderização no servidor e rotas por pastas
Publicação	Vercel, Netlify ou Cloudflare Pages, com a reescrita de rotas da seção 7.1

Anexo A. O contrato da API

Base: `https://startub.appmake.com.br/api`. Toda requisição leva `Accept: application/json`. As rotas marcadas com **token** exigem o cabeçalho `Authorization: Bearer <token>`; sem ele, ou com um token inválido, a resposta é **401** com `{ "message": "Unauthenticated." }`. As marcadas com **do dono** respondem **403** com `{ "message": "This action is unauthorized." }` para as outras contas. Todo evento traz o `user_id` de quem o criou, e toda inscrição, o de quem se inscreveu.

Método e caminho	Token	Corpo	Respostas
`POST /register`	não	name, email, password, password_confirmation	201 com `{ user, token }`, 422
`POST /login`	não	email, password	200 com `{ user, token }`, 422, 429 depois de 6 tentativas por minuto
`GET /user`	sim		200 com o usuário do token
`POST /logout`	sim		204, e o token deixa de valer
`GET /eventos`	sim		200 com a lista, em ordem de data
`GET /eventos/{id}`	sim		200, 404
`POST /eventos`	sim	titulo, descricao, local, data, vagas, valor	201, 422
`PATCH /eventos/{id}`	sim, do dono	qualquer campo do evento	200, 403, 404, 422

Método e caminho	Token	Corpo	Respostas
`DELETE /eventos/{id}`	sim, do dono		204, 403, 404
`GET /eventos/{id}/inscricoes`	sim, do dono do evento		200 com os inscritos, 403
`POST /eventos/{id}/inscricoes`	sim	nome, email, cpf	201, 404, 409, 422, 502
`GET /inscricoes/{id}`	sim, de quem se inscreveu		200 com o evento junto, 403, 404
`GET /ping`	não		200, para testar se a API está no ar

Anexo B. O mapa do projeto

Arquivo	Responsabilidade
`main.tsx`	providers: tema, cache, roteador, autenticação, minhas inscrições, avisos
`App.tsx`	as rotas, e quais ficam atrás do login
`tipos.ts`	os formatos dos dados da API
`services/api.ts`	a única função que fala com a API: endereço, token e erros
`services/auth.ts`, `eventos.ts`, `inscricoes.ts`	uma função por endereço da API
`schemas/`	as regras de cada formulário, em Zod
`lib/formatar.ts`	data e dinheiro no padrão brasileiro
`lib/erros.ts`	tradução dos erros da API e o envio de cada um para o campo certo
`contexts/`	o usuário logado e as minhas inscrições
`components/`	as peças: layout, cartão, estados, formulários, etiqueta, rota protegida
`components/ui/`	os componentes do shadcn
`pages/`	uma página por rota